Tema 5.

Servlets

Servlets

J2EE



Java

JSPs

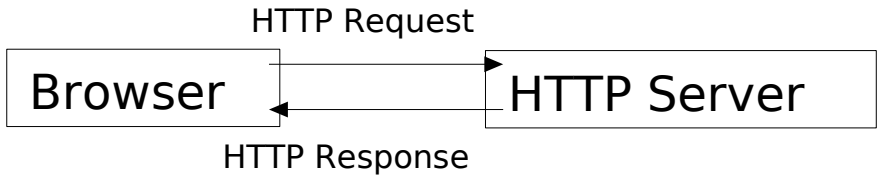
JDBC

Indice

- Introducción
- Servlets
- Ciclo de Vida de los Servlets
- Compartiendo recursos
  - ServletConfig
- Sesiones y Cookies
- Gestión de Errores
- Concurrencia en Servlet
- Acceso a Base de Datos

Introducción a los servlets

- Un servlet es un programa en Java que se ejecuta en un **servidor de aplicaciones**, mostrando al navegador el resultado de su ejecución.
- Los servlets actúan como capa intermedia entre:
  - Petición proveniente de un navegador Web u otro cliente HTTP.
  - Bases de Datos o aplicaciones en el servidor.
- Por tanto, un servlet puede recibir peticiones (normalmente HTTP) y generar una salida (normalmente HTML o XML).
- Otras opciones que permiten generar páginas web dinámicas son con los lenguajes ASP, PHP, JSP y Python.



Introducción a los servlets

- Un servidor de aplicaciones proporciona una estructura en tres capas que permite estructurar nuestro sistema de forma más eficiente.
  - Alta disponibilidad
  - Seguridad
  - Escalabilidad
  - Serialización
  - ….

Introducción a los servlets

- Existen un gran número de fabricantes con servidores de aplicaciones certificados J2EE (se verán en práctica)
- Lista: http://java.sun.com/j2ee/compatibility.html
  - **Libres**
    - **Tomcat**: Contenedor de Servlets y motor de JSPs. http://jakarta.apache.org/tomcat
    - **GlashFish:** Contenedor de Servlets y motor de JSPs. Sun/Oracle
    - **JBoss**: Servidor de aplicaciones J2EE. http://www.jboss.org
    - **Evidan JOnAS**: http://www.evidian.com/jonas
    - **OpenEJB**: http://openejb.sourceforge.net
  - **Comerciales**
    - **BEA WebLogic Server**
    - **IBM WebSphere**
    - **Oracle Application Server**
    - **.....**

Introducción a los Servlets

- Ejemplo: HolaMundo.java

```
package es.unex.cum.tw;

import java.io.*;
import javax.servlet.*; //jakarta.servlet
import javax.servlet.http.*; //jakarta.servlet.http

public class HolaMundo extends HttpServlet {
    protected void processRequest(HttpServletRequest request,
            HttpServletResponse response) throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        try {
            out.println("<html>");
            out.println("<head>");
            out.println("<title>Servlet HolaMundo</title>");
            out.println("</head>");
            out.println("<body>");
            out.println("<h1>Servlet HolaMundo</h1>");
            out.println("</body>");
            out.println("</html>");

        } finally {
            out.close();
        }
    }
}
```

Hereda de HttpServlet

Petición de entrada

Petición de salida

tipo de salida

flujo de salida

Salida HTML

Importante: En las últimas versiones de Servlet el paquete javax.servlet es jakarta.servlet y el paquete javax.servlet.http por jakarta.servlet.http .

Introducción a los Servlets

- Ejemplo: web.xml
  - Además de implementar el servlet, se debe configurar las URL para las que se utiliza el servlet (fichero web.xml)

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://java.sun.com/xml/ns/javaee" xmlns:web="http://java.sun.com/xml/ns/javaee/web-
app_2_5.xsd" xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd" version="3.0">
  <display-name>EjemplosClase</display-name>
  <servlet>
        <servlet-name>HolaMundo</servlet-name>
        <servlet-class>es.unex.cum.tw.HolaMundo</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>HolaMundo</servlet-name>
        <url-pattern>/HolaMundo</url-pattern>
    </servlet-mapping>
    <session-config>
        <session-timeout>
            30
        </session-timeout>
    </session-config>
    <welcome-file-list>
    <welcome-file>index.html</welcome-file>
  </welcome-file-list>
</web-app>
```

Nombre del servlet

Class a ejecutar

URL de la app

Configuración de la sesión

Introducción a los Servlets

- Ejemplo: web.xml
  - ~~Además de implementar el servlet, se debe establecer las URL para las que se utiliza el servlet (fichero web.xml)~~
  - Este paso se puede sustituir usando anotaciones. Esta configuración se realiza directamente en la clase que constituye el servlet. La anotación necesaria es @WebServlet. Esta anotación puede recibir varios parámetros.

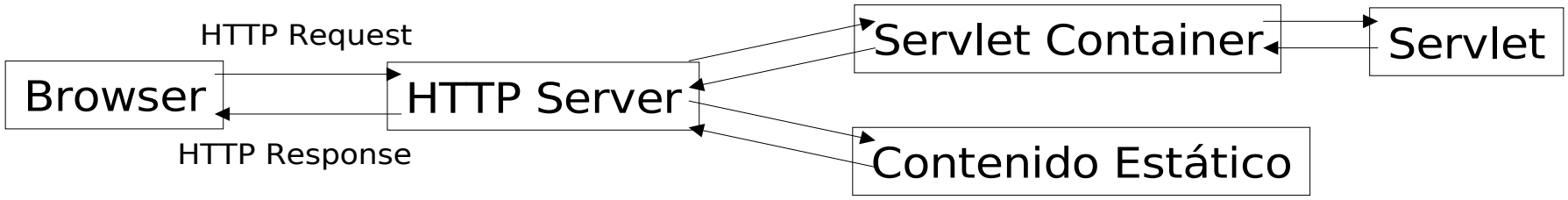
```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://java.sun.com/xml/ns/javaee" xmlns:web="http://java.sun.com/xml/ns/javaee/web-
app_2_5.xsd" xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd" version="3.0">
  <display-name>EjemplosClase</display-name>
  <servlet>
        <servlet-name>HolaMundo</servlet-name>
        <servlet-class>es.unex.cum.tw.HolaMundo</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>HolaMundo</servlet-name>
        <url-pattern>/HolaMundo</url-pattern>
    </servlet-mapping>
    <session-config>
        <session-timeout>
            30
        </session-timeout>
    </session-config>
    <welcome-file-list>
    <welcome-file>index.html</welcome-file>
  </welcome-file-list>
</web-app>
```



```
@WebServlet("/HolaMundoAnotacion")
public class HolaMundoAnotacion extends HttpServlet
{
            …..
}
```

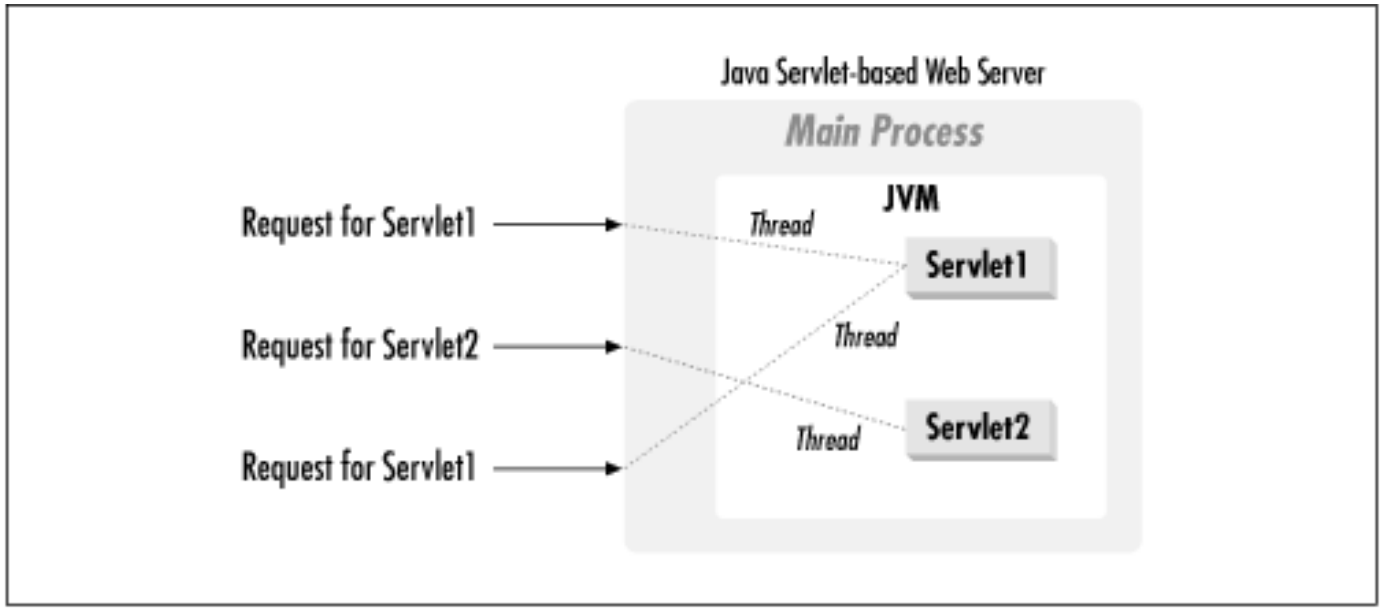
Servlets

- Los servlets son objetos que corren dentro del contexto de un servidor de aplicaciones y extienden su funcionalidad.
- La idea de servlet deriva de applet, pequeños programas escritos en Java que se ejecutan en un navegador web.
- Por contraposición, un servlet es un programa que se ejecuta en un servidor de aplicaciones, mostrando al navegador el resultado de su ejecución, por lo que, el uso más común de los servlets es generar páginas web de forma dinámica.
- Otras opciones que permiten generar páginas web dinámicas son con los lenguajes JSP, CGI, ...



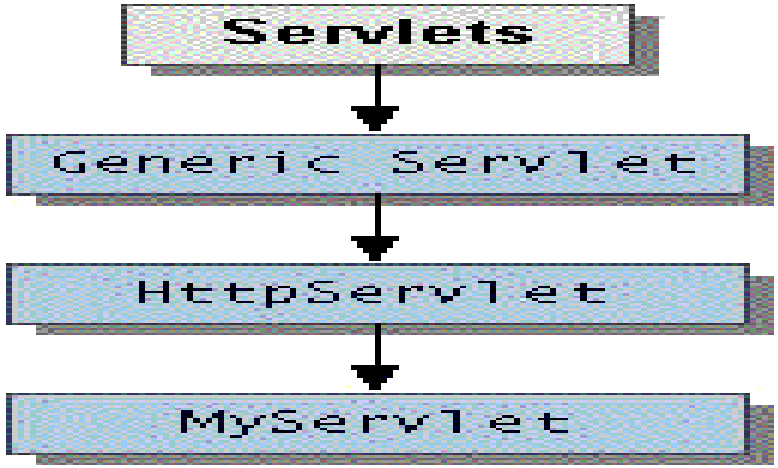
Servlets

- Función dentro del servidor
  - Los Servlets se ejecutan dentro del servidor a modo local. Son lanzados automáticamente por los clientes a través de la web pero no son ejecutados en la máquina cliente.
  - Los servlets recibirán una serie de peticiones y generarán información que se enviará a los clientes.

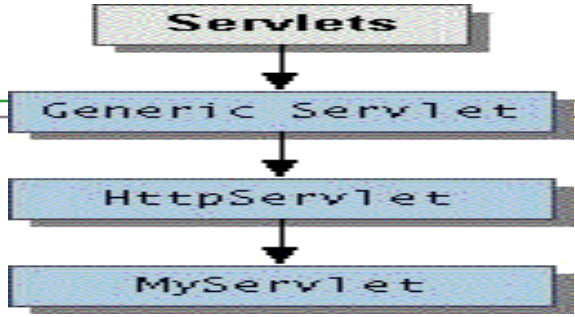


Servlets

- En Java hay un conjunto de clases que permiten implementar servlets.
- Recomendable ver Javadoc de **javax.servlet (jakarta.servlet).**
- En concreto, un servlet es un objeto Java que **implementa** la **interface javax.servlet.Servlet** o que **hereda** de alguna de su subclases.



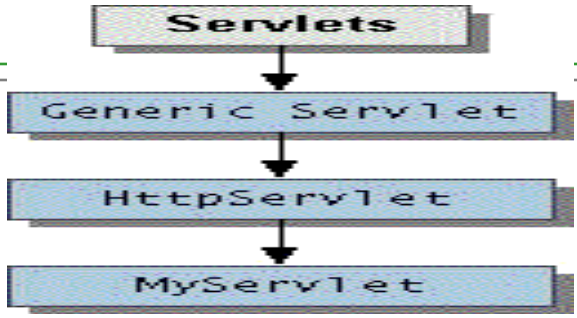
Jerarquía Servlet



- **Interface: javax.servlet.Servlet**
  - Este paquete contiene las clases e interfaces que son independientes de cualquier protocolo.

Método	Operación
public abstract void init (ServletConfig config) throws ServletException	Método que es llamado por el contenedor de servlets para indicar al servlet que debe inicializarse. Este método recibe como parámetro un objeto que implementa la interface ServletConfig
public abstract void Service (ServletRequest req,ServletResponse res) throws ServletException, IOException	Este método es llamado por el contenedor de servlets para cada petición del cliente de forma que asi se puede tratar la petición desde el servlet.
public abstract void destroy()	Este método es llamado por el contenedor de servlets para indicar al servlet que debe limpiarse, liberar cualquier recurso adquirido y estar lista para salir del servicio.
public abstract ServletConfig getServletConfig()	Información acerca del servlet
public abstract String getServletInfo()	La implementación de esta clase debe devolver información acerca del servlet versión, autor, copyright etc ...

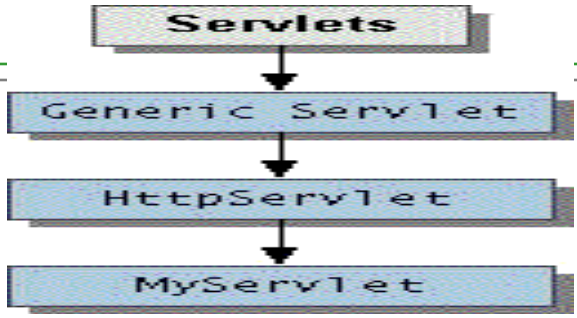
Jerarquía Servlet



- Paquetes

  - **javax/jakarta.servlet.GenericServlet**
    - ✓ Clase abstracta que implementa Servlet, ServletConfig,
    - ✓ Tiene implementaciones para todos los métodos de la interface Servlet excepto para el método service() .
    - ✓ Las peticiones se realizan mediante la interface **javax.servlet.ServletRequest** define métodos para el manejo de peticiones del cliente
    - ✓ La respuesta mediante la interface **javax.servlet.ServletResponse**

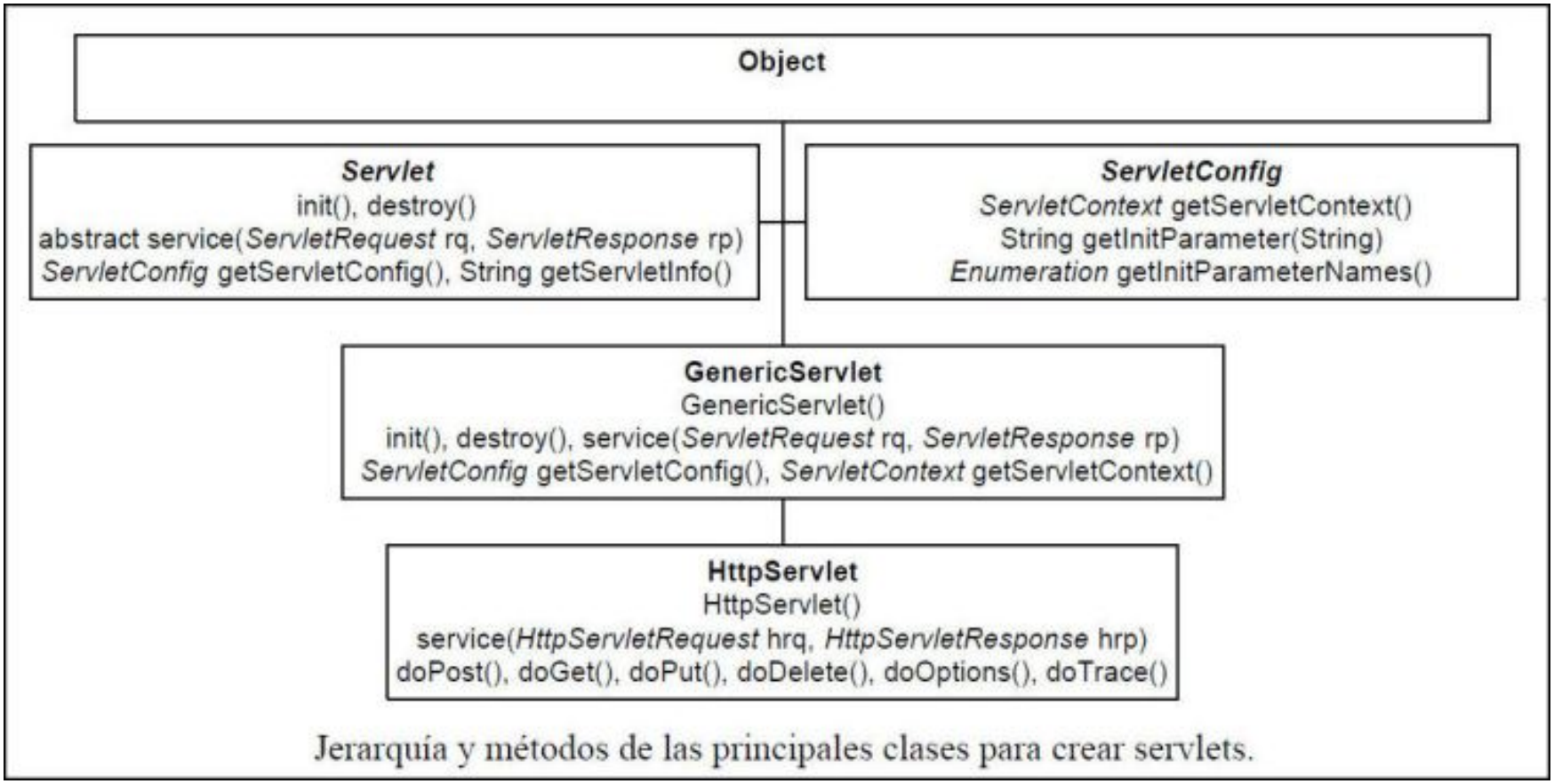
Jerarquía Servlet



- Paquetes

  - **javax.servlet.HttpServlet**
    - Proporciona la funcionalidad necesaria para desarrollar servlets HTTP.
    - HttpServlet extiende a la clase javax.servlet.GenericServlet y además añade los siguientes métodos
      - doPost: Manejar peticiones
      - doGet: Manejar peticiones
      - doPut: Método que coloca un documento directamente en el servidor.
      - doDelete: Método opuesto a PUT.
      - doOptions: Método puede ser usado para preguntar al servidor que métodos soporta o que opciones están activas para un servidor en particular.
      - doTrace: Método es usado para depuración. Retorna al cliente en contenido exacto de la respuesta.
    - HttpServletRequest y de javax.servlet.HttpServletResponse son los objetos utilizados para las peticiones y las respuestas

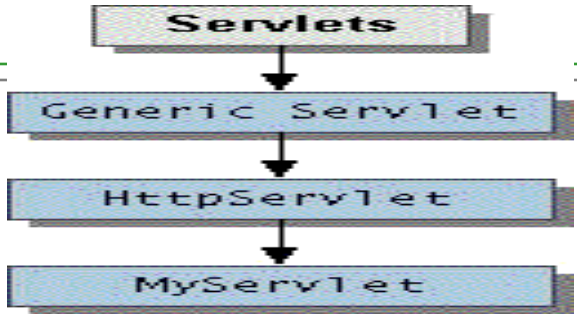
Jerarquía Servlet



Jerarquía y métodos de las principales clases para crear servlets.

Obtenido de http://jtagua.files.wordpress.com/2010/10/image128.jpg?w=730

Jerarquía Servlet



Paquetes: **HttpServlet**

```
public class Ejemplo extends HttpServlet {
    public Ejemplo() {
        super();
    }

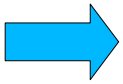
    public void init() throws ServletException {
        System.out.println("Starting servlet");
    }

    protected void service(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
        System.out.println("Call to Service method");
        super.service(req, resp);
    }

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
                        throws ServletException, IOException {
            System.out.println("Llamada al método doGet del servlet");
            response.getWriter().append("Served at: ").append(request.getContextPath());
   }

   protected void doPost(HttpServletRequest request, HttpServletResponse response)
                        throws ServletException, IOException {
            System.out.println("Llamada al método doPost del servlet");
            doGet(request, response);
   }
}
```

http://localhost:8080/PruebasTW/Ejemplo



Starting servlet
Call to Service method
Llamada al método doGet del servlet

Jerarquía Servlet

- Paquetes
  - **javax.servlet.HttpServletRequest (1/2)**

Método	Descripción
Object getAttribute(String name):	Devuelve el atributo asociado con el nombre que se le pasa con el argumento, o null si no hay ningún atributo asociado con dicho nombre.
java.util.Enumeration getAttributeNames():	Devuelve una enumeración que contiene los nombres de los atributos presentes en la petición.
String getParameter(String name):	Devuelve el parámetro asociado con el nombre que se le pasa como argumento, o null si dicho parámetro no existe
java.util.Map getParameterMap():	Devuelve un mapa con los parámetros de la petición.
String[] getParameterValues(String name)	Devuelve un array de cadenas de caracteres conteniendo todos los valores asociados con un determinado parámetro, o null si dicho parámetro no existe.
String getContextPath():	Devuelve el fragmento de la URL que indica el contexto de la petición. En un mismo servidor de aplicaciones puede haber desplegadas varias aplicaciones Java EE.

Jerarquía Servlet

- Paquetes
  - **javax.servlet.HttpServletRequest (2/2)**

Método	Descripción
Cookie[] getCookies():	Devuelve las cookies asociadas con esta petición. En un tema posterior hablaremos más en detalle de las cookies.
String getHeader(String name):	Devuelve el contenido asociado con la cabecera http especificada en el parámetro.
java.util.Enumeration<String> getHeaderNames():	Devuelve una enumeración conteniendo todas las cabeceras http de la petición.
java.util.Enumeration<String> getHeaders(String name):	Devuelve todos los valores asociados con una determinada cabecera http.
int getIntHeader(String name):	Devuelve el valor de una determinada cabecera http como un entero.
String getMethod():	Indica el método http empleado en la petición; por ejemplo GET, POST, o PUT.
httpSession getSession():	Devuelve la sesión asociada con esta petición. Si la petición no tiene una sesión asociada, crea una.
httpSession getSession(boolean create):	similar al método anterior, pero sólo crea la sesión si se le pasa el parámetro true.

Jerarquía Servlet

- Paquetes
  - **javax.servlet.HttpServletResponse**

Método	Descripción
java.io.PrintWriter getWriter()	Devuelve un objeto PrintWriter
void setContentType(String type):	Establece el tipo de contenido de la respuesta
void addCookie(Cookie cookie):	añade una cookie a la respuesta.
void addHeader(String name, String value):	añade una nueva cabecera, con el valor especificado
boolean containsHeader(String name):	Comprueba si la respuesta contiene o no una determinada cabecera.
String encodeRedirectURL(String url):	Prepara una URL para ser usada por el método sendRedirect.
String getHeader(String name):	Devuelve el valor de la cabecera http especificada.
java.util.Collection<String> getHeaderNames():	Devuelve los nombres de las cabeceras http de la respuesta.
java.util.Collection<String> getHeaders(String name):	Devuelve los valores asociados con una determinada cabecera http.
void sendError(int sc):	Envía al servidor una respuesta con el código de error
void sendRedirect(String location):	Redireciona al cliente a una determinada a URL.
ServletOutputStream getOutputStream():	Devuelve un objeto tipo ServletOutputStream;

Ejemplo 1: doGet y doPost (1/3) (GetPost.java):

```
package Clase;
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class GetPost extends HttpServlet {

  public void init(ServletConfig conf) throws ServletException {
    super.init(conf);
  }
  public void doGet(HttpServletRequest req, HttpServletResponse res) throws
ServletException, IOException  {
    res.setContentType("text/html");
    PrintWriter out = res.getWriter();
    out.println("<html>");
    out.println("<body>");
    out.println("<h1>Hola Mundo (llamada GET)</h1>");
    out.println("</body>");
    out.println("</html>");
  }

  public void doPost(HttpServletRequest req, HttpServletResponse res) throws
ServletException, IOException {
    res.setContentType("text/html");
    PrintWriter out = res.getWriter();
    out.println("<html>");
    out.println("<body>");
    out.println("<h1>Hola Mundo (llamada POST)</h1>");
    out.println("</body>");
    out.println("</html>");
  }
}
```

Código doGet

Código doPost

Ejemplo 1: doGet y doPost (2/3) (GetPost.java):

```
package es.unex.cum.tw;
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class HolaMundo extends HttpServlet {
   protected void processRequest(HttpServletRequest request, HttpServletResponse response)
   throws ServletException, IOException {
      response.setContentType("text/html;charset=UTF-8");
      PrintWriter out = response.getWriter();
      try {
         out.println("<html>"); out.println("<head>");
         out.println("<title>Servlet HolaMundo</title>");
         out.println("</head>"); out.println("<body>");
         out.println("<h1>Servlet HolaMundo</h1>");
         out.println("</body>");out.println("</html>");
      } finally {
         out.close();
      }
   }
   protected void doGet(HttpServletRequest request, HttpServletResponse response)
   throws ServletException, IOException {
      processRequest(request, response);
   }
   protected void doPost(HttpServletRequest request, HttpServletResponse response)
   throws ServletException, IOException {
      processRequest(request, response);
   }
}
```

Código a ejecutar

Llamada a processRequest

Ejemplo 1: doGet y doPost (3/3) (Get.html y post.html)

```
<html>
<body>
  <h1>Metodo GET</h1>
  <form method="GET" action="GetPost">
    <input type="submit">
  </form>
</body>
</html>
```




```
<html>
<body>
  <h1>Metodo POST</h1>
  <form method="POST" action="GetPost">
    <input type="submit">
  </form>
</body>
</html>
```



- Ejemplo 2: (Se conoce los parámetros) (Parametros.java)

```
package es.unex.cum.tw;
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class Parametros extends HttpServlet {

    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        PrintWriter out;
        String title = "Leyendo 3 parámetros";
        response.setContentType("text/html");
        out = response.getWriter();
        out.println("<HTML><HEAD><TITLE>");out.println(title);
        out.println("</TITLE></HEAD><BODY>");
        out.println("<H1 ALIGN=CENTER>" + title + "</H1>");
        out.println("<UL>");
        out.println("<LI>param1: " + request.getParameter("param1"));
        out.println("<LI>param2: " + request.getParameter("param2"));
        out.println("<LI>param3: " + request.getParameter("param3"));
        out.println("</UL>"); out.println("</BODY></HTML>");
        out.close();
    }
    public void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        doGet(request, response);
    }
}
```

http://localhost:8080/EjerciciosClase/Parametros?param1=Hola&param2=Adios&param3=hello

Servlet

- Ejemplo 3: Parámetros: Desconocemos que parámetros
  - String getParameter(name):
    - ✓Valor de parámetroindicado
  - Map<String, String[]> getParameterMap():
    - ✓Devuelve un mapa con los parámetros de la petición.
  - Enumeration<String> getParameterNames()
    - ✓Devuelve los nombres de los parámetros de la solicitud como una enumeración de cadenas
  - String[] getParameterValues(String name)
    - ✓Devuelve un array de cadenas de caracteres con todos los valores asociados a un determinado parámetro, o null si no existe.

```
Enumeration enum= request.getParameterNames();
// Bucle para procesar los parámetros.
while (enum.hasMoreElements()) {
  String name = (String) enum.nextElement();
  String value = request.getParameter(name)
}
```

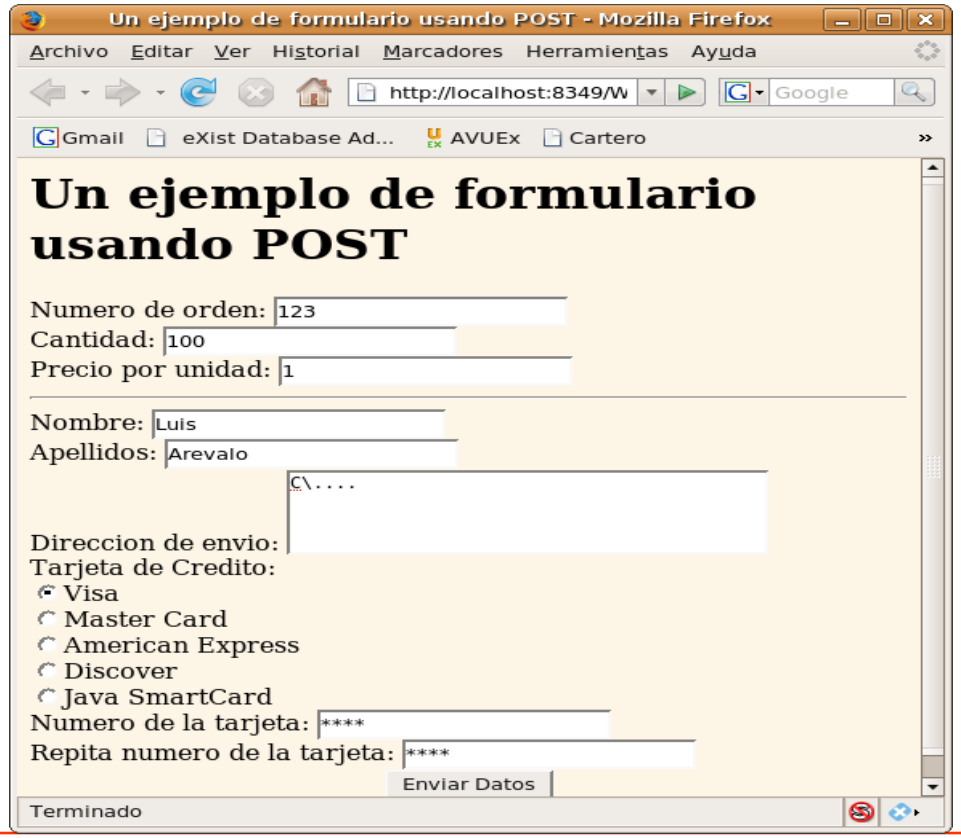
```
Map paramMap = request.getParameterMap();
// Bucle para procesar los parámetros.
Iterator it = paramMap.keySet().iterator();
while (it.hasNext()) {
  String name = (String) it.next();
  String value = request.getParameter(name)
}
```

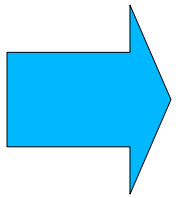
- Ejemplo 3: Parámetros: Desconocemos que parámetros
  - String[] getParameterValues(String name)
    - Devuelve un array de cadenas de caracteres con todos los valores asociados a un determinado parámetro, o null si no existe.

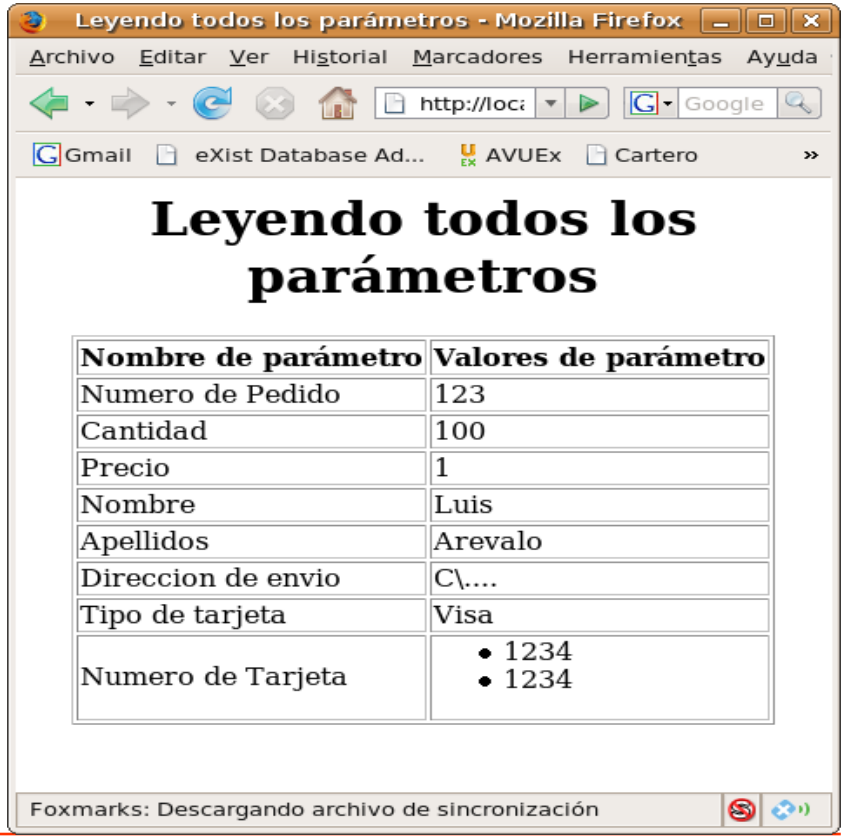
```
Enumeration nombresDeParam =request.getParameterNames();
while (nombresDeParam.hasMoreElements()) {
            String nombreParam = (String) nombresDeParam.nextElement();
             << nombreParam >>
            String[] valoresDeParam =request.getParameterValues(nombreParam);
            if (valoresDeParam.length == 1) {
                <<valoresDeParam[0] >>
            } else {
                 // Por ejemplo en un check o radiobutton
                for (int i = 0;i < valoresDeParam.length; i++) {
                    <<valoresDeParam[i])>>
            }
}
```

Servlet

- Los parámetros pueden ser usados en formularios
  - ShowParameters.java: Usando map
  - ShowParameters2.java: Usando enumeration
  - PostForm.html: Código HTML







- Ejemplo 4: Cabeceras.java
  - Se muestran las cabeceras de la petición http que ha recibido

```
protected void processRequest(HttpServletRequest request,
     HttpServletResponse response) throws ServletException, IOException {
   response.setContentType("text/html;charset=UTF-8");
   PrintWriter out = response.getWriter();
   try {
          out.println("<html>");
          out.println("<head>");
          out.println("<title>Servlet CabecerasServlet</title>");
          out.println("</head>");
          out.println("<body>");
          out.println("<h1>Cabeceras: </h1>");
          out.println("<ul>");
          Enumeration<String> nombresDeCabeceras = request.getHeaderNames();
          while (nombresDeCabeceras.hasMoreElements()) {
             String cabecera = nombresDeCabeceras.nextElement();
             out.println("<li><b>" + cabecera + ": </b>"
                    + request.getHeader(cabecera) + "</li>");
          }
          out.println("</ul>");
          out.println("</body>");
          out.println("</html>");
   } finally {
    out.close();
   }
}
```

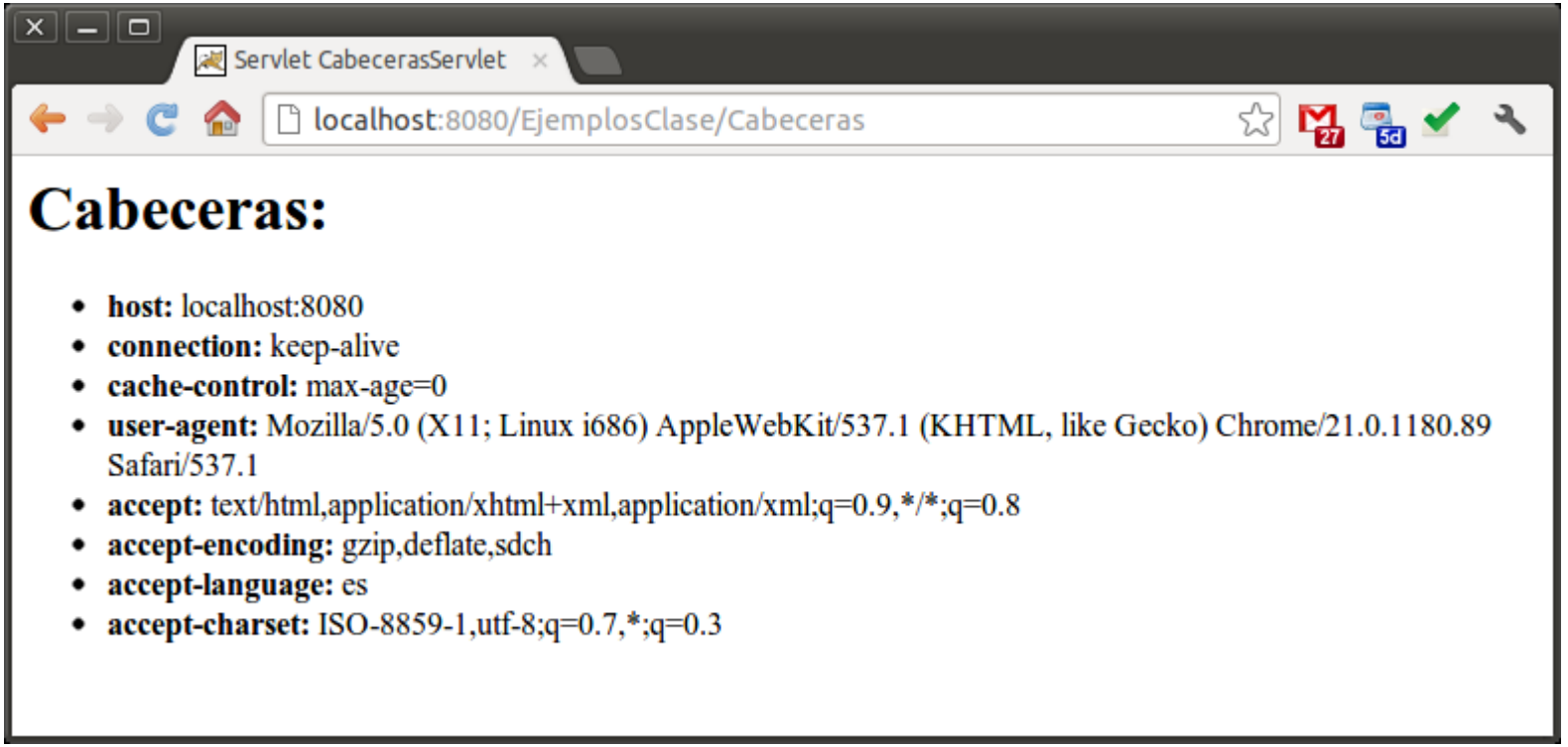
Enumeration de las cabeceras

Se recorren las cabeceras

Nombre

Contenido

- Ejemplo 4: Cabeceras
  - Se muestran las cabeceras de la petición http que ha recibido



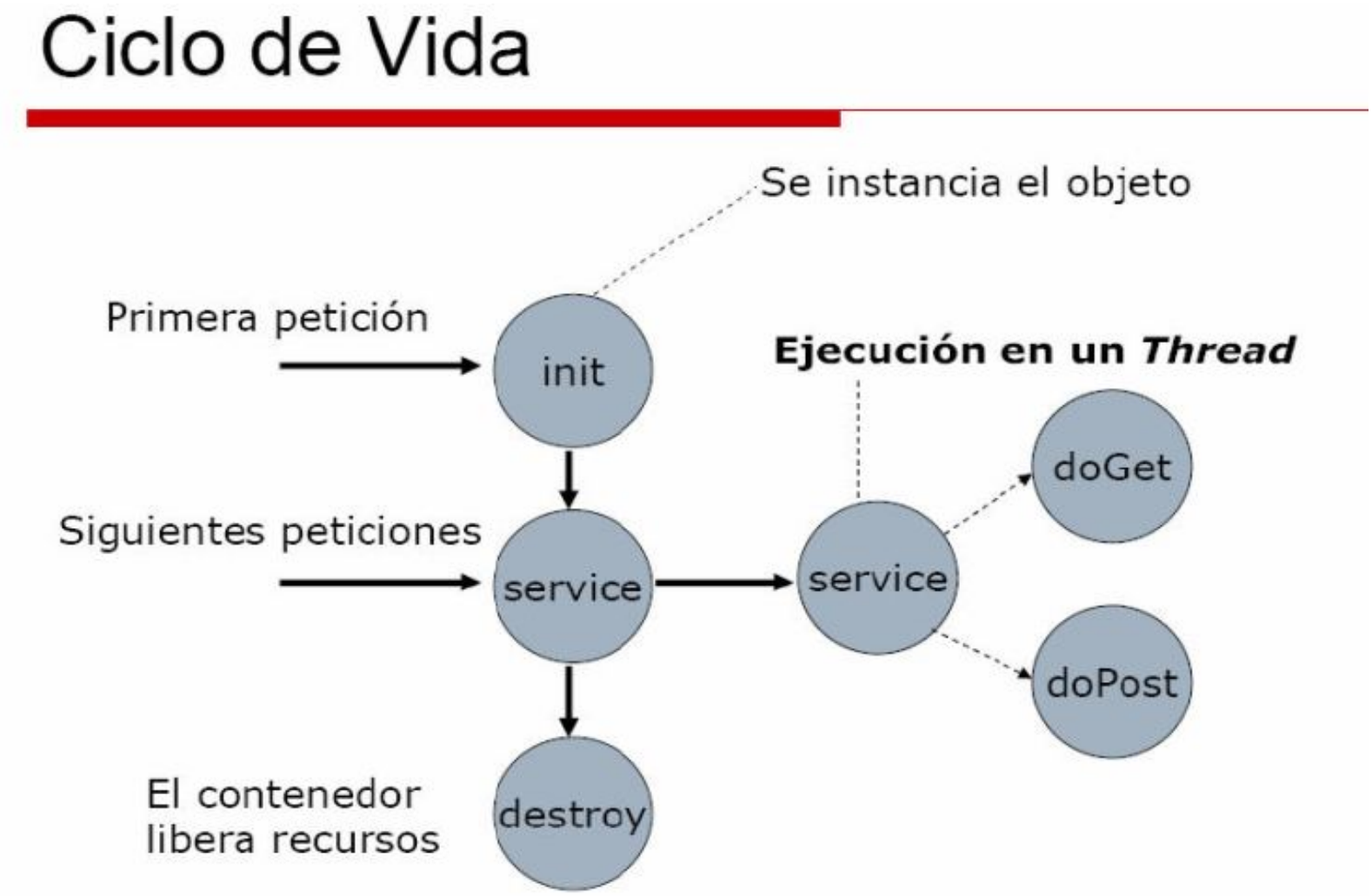
Servlets. Ciclo de Vida

- Los Servlets son gestionados por el contenedor.
- El contenedor es quien decide cuándo se debe de crear un determinado Servlet, y cuándo se debe destruir.
- El programador no controla directamente el ciclo de vida del Servlet, pero el API de Java si proporciona métodos para responder a los distintos eventos del ciclo de vida de un Servlet.

Servlets. Ciclo de Vida

- El ciclo de vida de un Servlet se divide:
  - El cliente solicita una petición a un servidor vía URL.
  - El servidor recibe la petición.
  - **init():** Si es la primera, se utiliza el motor de Servlets para cargarlo y se llama al método init(ServletConfig config).
    - Si ya está iniciado, cualquier petición se convierte en un nuevo hilo. Un Servlet puede manejar múltiples peticiones de clientes.
  - **service():**Se llama al método service() para procesar la petición devolviendo el resultado al cliente.
  - **destroy():** Cuando se apaga el motor de un Servlet se llama al método destroy(), que lo destruye y libera los recursos abiertos.
- Dos métodos adicionales:
  - **getServletInfo():** Obtiene información del servlet y puede ser sobreescrito por el programador
  - **getServletConfig():** Obtiene la configuración actual

Servlets. Ciclo de Vida



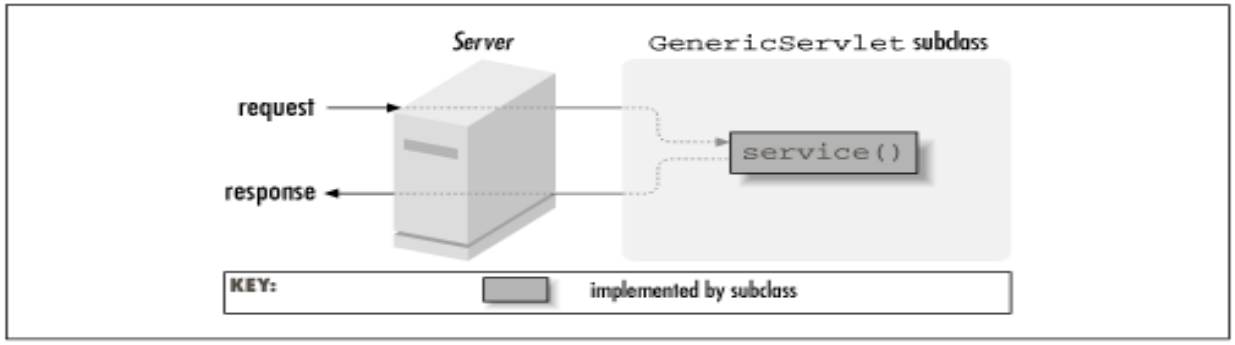
Obtenido de: http://jtagua.files.wordpress.com/2010/10/image215.jpg?w=730

Servlets. Ciclo de Vida

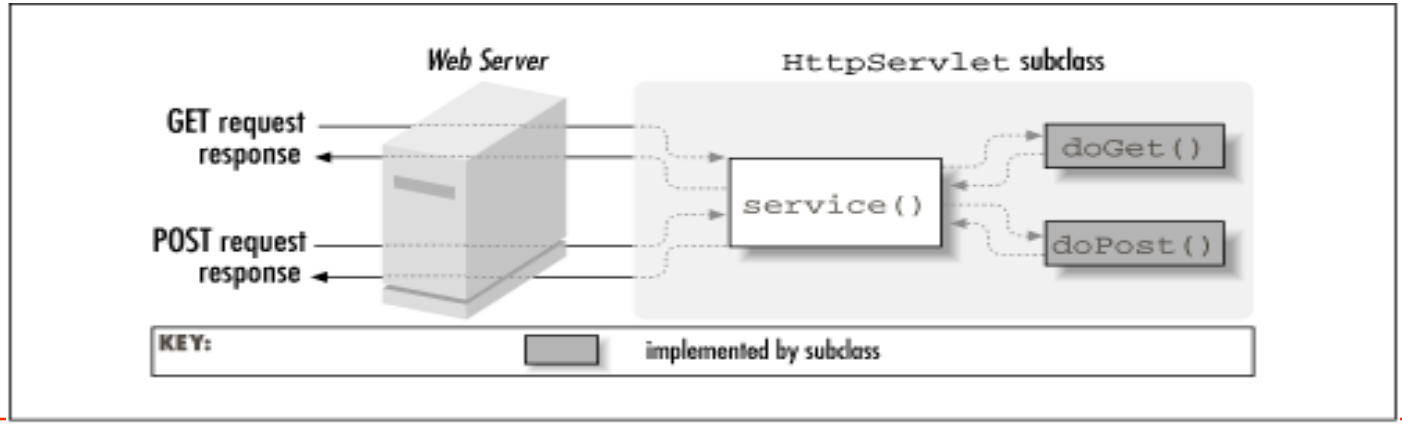
- Get vs Post
  - Get: Los parámetros del formulario están incluidos la url que se utiliza para invocar el servlet
  - Post: Los parámetros se almacenan en un buffer especial del servidor.
- La implementación por defecto del método service es capaz de determinar el tipo de petición HTTP que en un momento dado recibe el servlet.
  - Una vez identificada llama al método doGet o doPost.
  - Como, en la mayoría de los casos, seremos nosotros quien programe el formulario que llame al servlet, sabremos que tipo de llamada se hará, por lo que podemos optar por redefinir uno sólo de los métodos.
  - En el caso de que no lo supiéramos se deben implementar los métodos doGet y doPost.

Servlets. Ciclo de Vida

- Cuando un servlet tienen activo uno de los métodos su funcionamiento es el siguiente:



- En cambio, cuando en un servlet se activan los métodos GET y POST simultáneamente su funcionamiento es similar al mostrado por la figura.

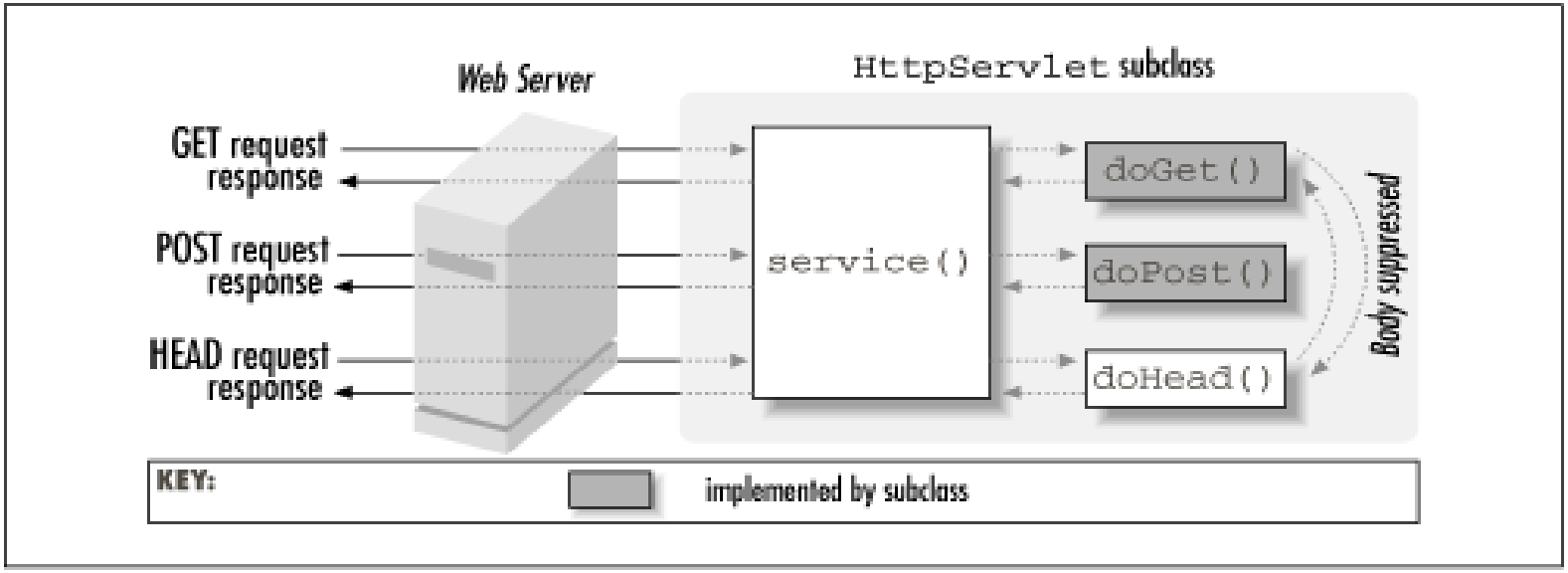


Servlets. Ciclo de Vida

- Existen otros métodos para los servlets menos utilizados pero igual de importantes. Estos métodos son:
  - HEAD: Método enviado por el cliente cuando necesita solo la respuesta de las cabeceras, determinar el tamaño del documento que generará el servlet, tiempo de modificación o viabilidad del documento.
  - PUT: Método que coloca un documento directamente en el servidor.
  - DELETE: Método opuesto a PUT.
  - TRACE: Método es usado para depuración. Retorna al cliente en contenido exacto de la respuesta.
  - OPTIONS: Método puede ser usado para preguntar al servidor que métodos soporta o que opciones están activas para un servidor en particular.

Servlets. Ciclo de Vida

- Los métodos pueden actuar unos con otros y cambiar información, como se muestra en la figura.

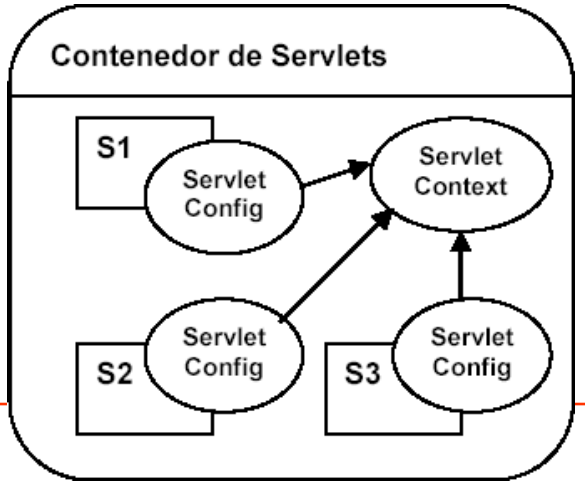


Contexto

- Cuando un servlet maneja una petición, el procesamiento puede requerir el uso de información procedente de diferentes contextos.
  - El primer contexto es la **aplicación**. La información a nivel de aplicación es accesible para procesar todas las consultas de todos los clientes. La información a nivel de aplicación nunca debe ser específica del cliente.
  - El segundo contexto es la **sesión**. La información contenida en la sesión es accesible para procesar todas las peticiones del **mismo cliente**.
  - El tercer contexto es **la petición**. La información en la petición permite proporcionar al cliente una respuesta personalizada para la petición en cuestión. La información introducida en un formulario está relacionada con una petición.

Contexto de Aplicación

- Durante la inicialización del servlet se puede configurar diversos parámetros
  - Autenticación de usuarios si no hay una BD
  - Configuración de JDBC (en lugar de incluirlo en código)
  - Etc
- Esta configuración se realiza en el fichero (web.xml).
- Se puede establecer a dos niveles
  - **Por cada servlet**: la configuración solo afecta a ese servlet
  - **Por el contexto**: Afecta a todos los servlets definido



Contexto de Aplicación

- Por cada Servlet, se debe indicar el nombre del parámetro y su valor. Se utiliza por ejemplo

```
<servlet>
        <servlet-name>Autentificacion</servlet-name>
        <servlet-class>es.unex.cum.tw.Autentificacion</servlet-class>
        <init-param>
                <param-name>usuario</param-name>
                <param-value>administrador</param-value>
        </init-param>
        <init-param>
                <param-name>password</param-name>
                <param-value>prueba</param-value>
        </init-param>
</servlet>
```

- Posteriormente se procederá a la carga de esta información

```
public void init(ServletConfig config) throws ServletException {
        super.init(config);   //Llamada al constructor de la CB
        nombre = config.getInitParameter("usuario"); //Se recupera el valor de usuario
        password = config.getInitParameter("password"); //Se recupera el valor de pass
}
```

Contexto de Aplicación

- Ejercicio 5: CargaConfiguracion (CicloVida/Autenticacion.java)

```
public class Autenticacion extends HttpServlet {
private String nombre = null;
private String password = null;

public void init(ServletConfig config) throws ServletException {
          super.init(config);
          nombre = config.getInitParameter("usuario");
          password = config.getInitParameter("password");
}

public void processRequest(HttpServletRequest req, HttpServletResponse res)
          throws IOException {
   res.setContentType("text/html");
   PrintWriter out= res.getWriter();
   out.println("<html><head><title>");out.println("Autenticacion</title><body>");

   String nombreRecibido = req.getParameter("user");
   String passRecibido = req.getParameter("password");

   if (nombreRecibido==null || passRecibido==null) {
          out.println("<h1>Datos no válidos<h1>");
   } else if (nombreRecibido.equals(nombre) && passRecibido.equals(password)) {
          out.println("<h1>Usuario autenticado<h1>");
   } else out.println("<h1>Usuario no autenticado<h1>");
   out.println("</body></html>");
   out.close();
}
```

Se recupera el nombre y pass en la Inic

Se comprueba si el Nombre y pass coinciden

http://localhost:8080/EjemplosClase/Autenticacion?user=administrador&password=prueba

Contexto de Aplicación

- En un servlet, se puede acceder al contexto de la aplicación a través del método **getServletContext**() que devuelve un objeto de tipo **ServletContext**.
- Esta formada por los siguientes métodos
  - getContextPath():
    - Devuelve una cadena de caracteres correspondiente a la URL que permite acceder a la aplicación.
  - String getInitParameter(String name):
    - devuelve el parámetro de inicialización asociado con el nombre
  - java.util.Enumeration<String> getInitParameterNames():
    - devuelve los nombres de los parámetros de inicialización. Si no hay ningún parámetro, la enumeración que devuelve estará vacía.
  - String getServletName():
    - devuelve el nombre de esta instancia de Servlet.

Contexto de Aplicación

- Parámetros establecidos en el contexto

```
<web-app xmlns="http://java.sun.com/xml/ns/j2ee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee
http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd" version="2.4">
  <context-param>
            <param-name>usuarioContext</param-name>
            <param-value>administrador</param-value>
  </context-param>
  <context-param>
            <param-name>passwordContext</param-name>
            <param-value>prueba</param-value>
  </context-param>
```

```
public class AutenticacionContext extends HttpServlet {
    private String nombre = null;
    private String password = null;

    public void init(ServletConfig config) throws ServletException {
            super.init(config);     //Se debe inicializar la clase base
            ServletContext context = getServletContext(); //Se recupera el contexto
            nombre = context.getInitParameter("usuarioContext");
            password = context.getInitParameter("passwordContext");
     }
     …
}
```

http://localhost:8080/EjemplosClase/AutenticacionContext?user=administrador&password=prueba

Contexto de Aplicación

- Tanto en el contexto de aplicación como en el contexto de servlet se puede establecer un fichero de propiedades, para en él tener todos las contraseñas en lugar de tener múltiples etiquetas en el web.xml
- De este modo, se prefiere usar un fichero independiente, denominado fichero de propiedades
  - Estos fichero de configuración se establecen como parámetro de configuración del Servlet
  - A continuación con el método getResourceAsStream() se obtiene la información de dicho fichero
  - El fichero es un fichero de propiedades, y se lee empleando la clase java.util.Properties. El contenido de este fichero:
    - usuario1 = administrador
    - password1 = prueba
    - usuario2 = luis
    - password2 = prueba

Contexto de Aplicación

- Fichero compartido: Fichero de propiedades: Pasos
  - Configurar en web.xml el fichero de propiedades

```
<servlet>
   <servlet-name>Autenticacion3</servlet-name>
   <servlet-class>es.unex.cum.tw.contexto.Autenticacion3</servlet-class>
   <!-- Como servlet -->
   <init-param>
          <param-name>ficheroUsuarios</param-name>
          <param-value>WEB-INF/ficheroUsuarios.prop</param-value>
   </init-param>
</servlet>
```

  - A nivel de contexto

```
<context-param>
         <param-name>ficheroUsuarios</param-name>
         <param-value>WEB-INF/ficheroUsuarios.prop</param-value>
</context-param>
```

Contexto de Aplicación

- Fichero compartido: Fichero de propiedades: Pasos (1/2)
  - Autenticacion3.java

```
public class Autenticacion3 extends HttpServlet {
  private String user = null;
  private String password = null;
  public void init(ServletConfig config) throws ServletExcept... {
    super.init(config);
     ServletContext context = getServletContext();
     String ficheroUsuarios = config.getInitParameter("ficheroUsuarios");     //Servlet
     //String ficheroUsuarios = context.getInitParameter("ficheroUsuarios"); //Contexto
     InputStream is = context.getResourceAsStream(ficheroUsuarios);
     Properties ficheroPropiedades = new Properties();
    try {
          ficheroPropiedades.load(is);
          user= ficheroPropiedades.getProperty("UserAdmin");
          password = ficheroPropiedades.getProperty("PasswordAdmin");
     } catch (Exception ex) {

     }
}
```

Se obtiene el path de FicherosUsuarios en web.xml

Flujo al fichero

Carga del fichero

Carga del datos

Contexto de Aplicación

- El mismo proceso realizado con anotaciones (nada de información en el fichero web.xml)

```
@WebServlet(name="Autenticacion3_Anotaciones",
        value={"/Autenticacion3_Anotaciones"},
        initParams={@WebInitParam(
                name="ficheroUsuarios",
                value = "WEB-INF/ficheroUsuarios.prop")})

public class Autenticacion3_Anotaciones extends HttpServlet {
   private String user = null;
   private String password = null;

   public void init(ServletConfig config) throws ServletException {
        super.init(config);
        ServletContext context = getServletContext();
        String ficheroUsuarios = context.getInitParameter("ficheroUsuarios");
        InputStream is = context.getResourceAsStream(ficheroUsuarios);
        Properties ficheroPropiedades = new Properties();
        try {
            ficheroPropiedades.load(is);
            user = ficheroPropiedades.getProperty("UserAdmin");
            password = ficheroPropiedades.getProperty("PasswordAdmin");
        } catch (Exception ex) {
           ex.printStackTrace();
           }
}
```

Compartiendo recursos

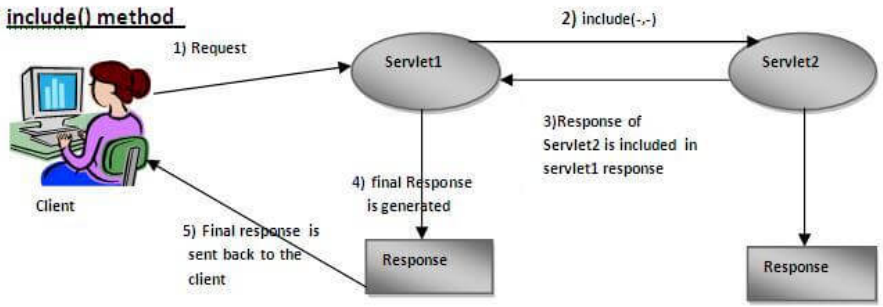
- El tratamiento de una **petición HTTP** puede compartirse entre varios servlets o un servlet y una página JSP. Para otro servlet está prohibido instanciarlo nosotros mismos, sino que debemos pasar por el contenedor.
- Hay tres soluciones posibles para obtener la colaboración entre recursos de una petición HTTP.
  - Utilización del RequestDispatcher
    - Se redirige la petición de un recurso a otro internamente, dentro de la misma aplicación Web.
  - Redirección
    - El servidor envía una respuesta al cliente con un código 3xx y la URI a la que este debe enviar la petición. Dos peticiones HTTP: la original y la dirigida a la nueva URI.
  - Filtros
    - Se usan para añadir un tratamiento para que se realice en una petición HTTP antes de que el servlet la reciba o en una respuesta.

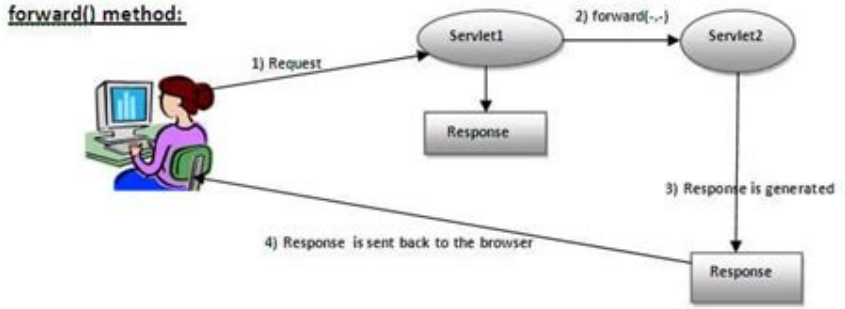
Compartiendo recursos: RequestDispatcher

- Utilización del RequestDispatcher
  - Se redirige la petición de un recurso a otro internamente, dentro de la misma aplicación Web.
  - El recurso de la última redirección devuelve al cliente la respuesta HTTP.
  - Para ello hay que obtener un objeto RequestDispatcher del contexto de aplicación mediante el métod getRequestDispatcher()
  - Este método recibe como parámetro una cadena de caracteres que identifiquen el recurso hacia el que la petición HTTP tiene que transferirse. Tiene que empezar con el carácter "/" para indicar que la ruta se expresa desde la raíz de la aplicación.
  - Incluso se puede solicitar un recurso disponible en otra aplicación albergada en el mismo servidor.

Compartiendo recursos: RequestDispatcher

- Utilización del RequestDispatcher. Dos soluciones
  - Include: Permite escribir contenido en el objeto ServletResponse a continuación de lo que se haya escrito ya por parte de nuestro servlet.
  - Forward: sólo podrá ser invocado cuando todavía no se ha escrito nada en la respuesta del servlet. Este método transfiere la petición a otro recurso para completar el procesamiento.



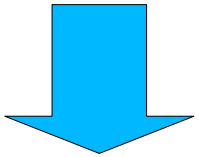



```
RequestDispatcher rd =request.getRequestDispatcher ("/HolaMundo") ;
rd.forward(request,response) ;
```

Compartiendo recursos: RequestDispatcher

Forward

```
@WebServlet(name="HolaMundoCompartir",urlPatterns={"/Compartir/HolaMundo"})
public class HolaMundo extends HttpServlet {
   public void doGet(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException {
          RequestDispatcher rd =req.getRequestDispatcher ("/Compartir/Forward") ;
          rd.forward(req,res) ;
   }
   public void doPost(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException {
          doGet(req, res);
   }
}
```

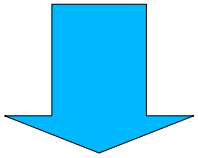


```
@WebServlet(name="Forward",urlPatterns={"/Compartir/Forward"})
public class Forward extends HttpServlet {
   public void doGet(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException {
     res.setContentType("text/html;charset=UTF-8");
     PrintWriter out = res.getWriter();
     try {
           out.println("<h1>Servlet HolaMundo</h1>");
           out.println("<h1>He realizado forward</h1>");
     } finally {
            out.close();
     }
   }
   public void doPost(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException {
       doGet(req, res);
   }
}
```

Compartiendo recursos: RequestDispatcher

Include

```
@WebServlet(name="HolaMundoInclude",urlPatterns={"/Compartir/HolaMundoInclude"})
public class HolaMundoInclude extends HttpServlet {
  private static final long serialVersionUID = 1L;
  public void doGet(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException {
      PrintWriter pw=  res.getWriter();
      pw.write("<b>Primer servlet</b> <br>");
      RequestDispatcher rd =req.getRequestDispatcher ("/Compartir/Include") ;
      rd.include(req,res) ;
  }
  public void doPost(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException {
     doGet(req, res);
  }
}
```



```
@WebServlet(name="Include",urlPatterns={"/Compartir/Include"})
public class Include extends HttpServlet {
  private static final long serialVersionUID = 1L;
  public void doGet(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException {
      PrintWriter pw=  res.getWriter();
      pw.write("<b>Segundo servlet</b> <br>");
  }
  public void doPost(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException {
      doGet(req, res);
  }
}
```

Compartiendo recursos: RequestDispatcher

- Hay dos métodos de dispatcher:
  - **getRequestDispatcher**(ruta) donde ruta se refiere a la url relativa o absoluta a la que responde el servlet destino.

```
RequestDispatcher rd =request.getRequestDispatcher ("/principal") ;
rd.forward(request,response) ;

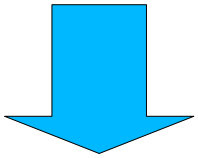
RequestDispatcher rd =request.getNamedDispatcher ("principal") ;
rd.forward(request,response) ;
```

  - **getNamedDispatcher**(ruta): donde ruta se refiere al nombre del servlet registrado en el web.xml o en las anotaciones @Web. No es recomendable su uso porque suele generar bastante problema.

Compartiendo recursos: RequestDispatcher

- getRequestDispatcher
  - El objeto de la petición (ServletRequest) de los recursos origen y destino de la redirección es el mismo.
  - Por este motivo es muy frecuente que se puedan añadir nuevos parámetros como atributos a la petición.

```
@WebServlet(name="HolaMundoAtributo",urlPatterns={"/Compartir/HolaMundoAtributo"})
public class HolaMundoAtributo extends HttpServlet {
   public void doGet(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException {
           int x= 10;
           req.setAttribute("valor",x);
           RequestDispatcher rd =req.getRequestDispatcher ("/Compartir/Atributo") ;
           rd.forward(req,res) ;
   }
```



```
@WebServlet(name="Atributo",urlPatterns={"/Compartir/Atributo"})
public class Atributo extends HttpServlet {
  public void doGet(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException {
     String valor=(String) req.getAttribute("valor");
     PrintWriter out = res.getWriter();
     out.println(valor);
  }
}
```

Compartiendo recursos: RequestDispatcher

- Ejemplo de sendDirect y getRequestDispatcher

```
protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        String user = request.getParameter("user");
        String pass = request.getParameter("pass");
        User u=db.getUser(user,pass);
        if (u==null){ response.sendDirect("loginFailed");}
        session.setAttribute("user",u);
        RequestDispatcher rd=request.getRequestDispatcher("principal");
        rd.forward(request,response);
}

----------Principal.java
protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException {
        String value=req.getAttribute(“user”);
        …..
}
```

Compartiendo recursos: SendDirect

- Redirecciones HTTP (sendRedirect):
  - El servidor envía una respuesta al cliente con un código 3xx y la URI a la que este debe enviar la petición.
  - Dos peticiones HTTP: la original y la dirigida a la nueva URI.

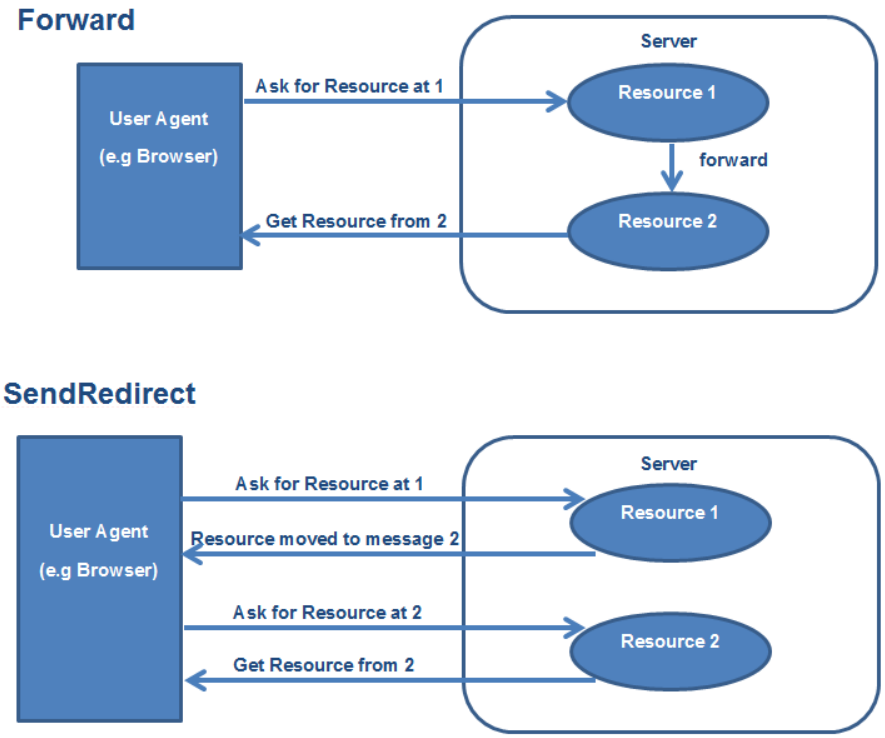


Imagen obtenida de https://i.stack.imgur.com/a3pCn.png

Compartiendo información entre Servlet

- SendDirect
  - Fuerza el envío de una respuesta HTTP de redirección al cliente.
  - Será el cliente, quien tras recibir esta respuesta solicite la página a la que debe redirigirse.
  - El control retorna al finalizar el método sendRedirect, por lo que conviene que sea lo último que se ejecuta.

```
// Redirección con URI absoluta
response.sendRedirect ("http://cum.unex.es/") ;
// Redirección con URI relativa a la de la petición actual
response.sendRedirect ("login.html") ;
```

Seguimiento del usuario: Sesiones y Cookies

- El protocolo http es un protocolo sin estado.
  - Una petición de un determinado recurso al servidor, el servidor responde sirviendo ese recurso.
  - Si a continuación se realiza una nueva petición al mismo/otro recurso, el servidor no es consciente de si ha o no servido algún otro recurso en el pasado al usuario, o de qué recursos ha servido.
- Es necesario realizar un seguimiento de trayectorias de los usuarios. Por ejemplo en un Carrito de la compra:
  - Cuando se añaden artículos a su cesta, ¿cómo sabe el servidor lo que hay ya en sus cestas de la compra?
  - Cuando se confirma el pedido, ¿cómo sabe el pedido cuál de las cestas de la compra previamente creadas es la suya?
  - …..

Servlets: Sesiones y Cookies

- Es necesario realizar un seguimiento de trayectorias de los usuarios. Formas:
  - Cookies
  - Sesiones en Java (Session Tracking) → clase HttpSession
  - Reescritura de URLs
  - Campos ocultos

Servlets: Sesiones y Cookies

- Cookies: Objetos de la clase Cookie
  - Representa un dato enviado por el servidor con destino al navegador. Cuando el navegador analiza esta respuesta HTTP, extrae la o las cookies y las almacena en el ordenador cliente.
  - Se almacenan en los **equipos de los clientes**
    - ✓ El cliente debe soportar cookies
    - ✓ Pueden ser desactivadas por el cliente
    - ✓ El navegador es el encargado de almacenarlas
  - Se transmiten en las cabeceras cuando se realiza la comunicación HTTP
  - Las cookies se implementan como una colección y se usan mediante los objetos integrados HttpServletRequest y HttpServletResponse
  - Web con ejemplos: http://www.java2s.com/Tutorial/Java/0400__Servlet/0080__Cookie.htm

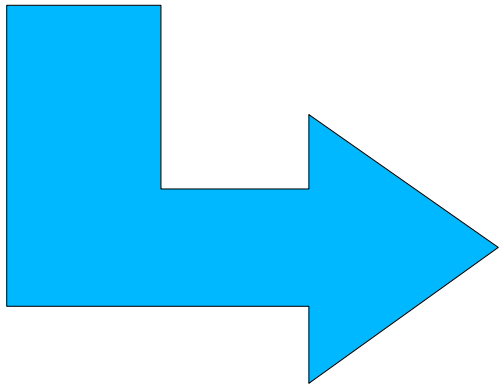
Servlets: Sesiones y Cookies

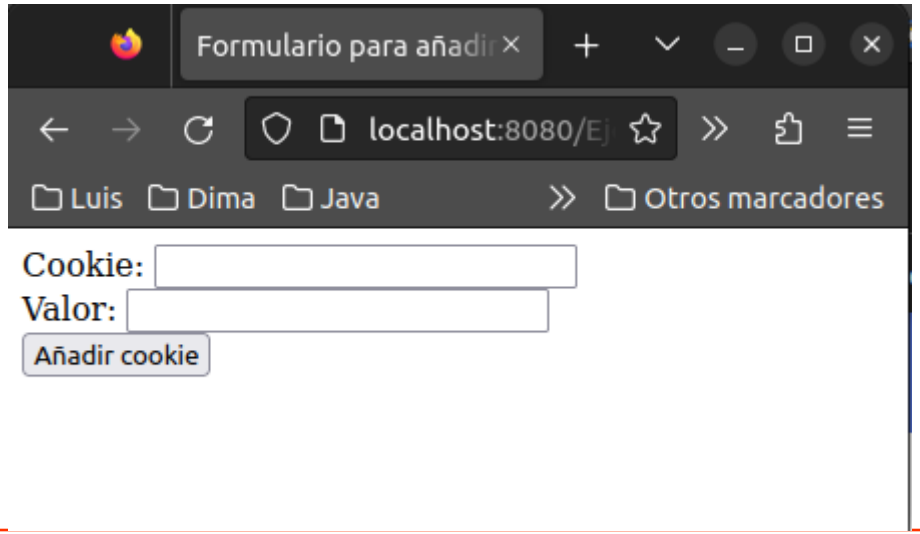
- Cookies: Objetos de la clase Cookie
  - **Cookie(String name, String value): construye una cookie con el nombre especificado y el valor. No se puede modificar el nombre una vez creado.**
  - int getMaxAge(): devuelve la edad máxima de esta cookie, medida en segundos.
  - void setMaxAge(int expiry): establece la máxima vida de esta cookie, medida en segundos. 0 → cookie será eliminada
  - **String getName(): devuelve el nombre de la cookie.**
  - String getPath(): devuelve la ruta del servidor a la cual el navegador web devolverá la cookie.
  - void setPath(String uri): especifica la ruta a la cual la cookie deberá ser devuelta por el navegador web.
  - **String getValue(): El valor actual de la cookie. Es posible modificar el valor de la cookie una vez ha sido establecida.**
  - **void setValue(String newValue): establece un nuevo valor.**

Servlets: Sesiones y Cookies

Cookies: Objetos de la clase Cookie (AddCookie.html)

```
<html>
  <head>
     <meta content="text/html; charset=ISO-8859-1" http-equiv="content-type">
     <title>Formulario para añadir cookies</title>
  </head>
  <body>
        <form method="get" action="/EjemplosClase/CookieServlet" name="datos">
              Cookie: <input name="cookie"><br>
              Valor:   <input name="valor"><br>
           <button>Añadir cookie</button>
        </form>
  </body>
</html>
```





Servlets: Sesiones y Cookies

Cookies: Clase Cookie (Sesiones/CookieServlet.java)

```
@WebServlet(name = "Galletita", urlPatterns = { "/Seguimiento/Cookie" })
public class Galletita extends HttpServlet {
  public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
     String nombreNuevaCookie = request.getParameter("cookie");
     String valor = request.getParameter("valor");
     int duracion=10000;
     PrintWriter out = response.getWriter();
     try {
            out.println("El contenido de tu sesion en cookie es:");
            if (nombreNuevaCookie != null && valor != null && !nombreNuevaCookie.equals("")) {
                Cookie nuevaCookie = new Cookie(nombreNuevaCookie,valor);
                nuevaCookie.setMaxAge(duracion);
                response.addCookie(nuevaCookie);
                out.println(nuevaCookie.getName() + " con valor " + nuevaCookie.getValue() +
                                    " y  fecha de expiracion: "+ nuevaCookie.getMaxAge());
            }
            out.println("Mostramos todas las cookies");
            Cookie[] todasLasCookies = request.getCookies();
            if (todasLasCookies != null) {
                for (Cookie cookie : todasLasCookies) {
                        out.println("Nombre: "+cookie.getName()+ " valor: "+cookie.getValue());
                }
            }
     } catch (Exception e) {
            response.sendRedirect ("error.html") ;
     }
  }
}
```

Se obtienen los parámetros

Se crea la cookie con los datos de Ent.

Mostramos todas las cookies

Servlets: Sesiones y Cookies

- Sesiones en Java (Session Tracking)
  - Los objetos de la sesión se guardan en el servidor
  - Se pueden guardar objetos arbitrarios dentro de una sesión
  - Las sesiones se asocian automáticamente al cliente vía Cookies o Reescritura de URLs
    - Como una caja negra para el cliente, el sistema se encarga de utilizar el método apropiado para mantener la sesión, bien mediante cookies o mediante reescritura de URLs
  - Clase httpSession
    - Mediante el método request.getSession() se obtiene un objeto HttpSession que se deberá procesar
    - Una sesión es un objeto Java que estará ligado de modo unívoco al visitante, cada vez que acceda, se recuperará.
    - HttpSession proporciona métodos que almacenan y recuperan:
      - Propiedades de Sesión estándar, como un identificador de sesión.
      - Datos de la aplicación, un mapa de pares (clave, valor), donde las claves son cadenas de caracteres y los valores puede ser cualquier objeto

Servlets: Sesiones y Cookies

Sesiones en Java (Session Tracking)

Método	Descripción
Object getAttribute(String name):	Devuelve un objeto java almacenado en la sesión del usuario, o null si no hay ningún objeto asociado con el nombre
void setAttribute(String name, Object value):	Añade a la sesión el objeto que se le pasa como segundo parámetro, siendo la clave asociada el primer parámetro.
Enumeration getAttributeNames():	devuelve una enumeración de cadenas de caracteres con todos los nombres de todos los objetos almacenados en la sesión del usuario.
long getCreationTime():	devuelve el instante en el que fue creada la sesión medido en milisegundos desde el uno de enero de 1970.
String getId():	devuelve una cadena de caracteres conteniendo un identificador único para la sesión.
long getLastAccessedTime():	Instante de tiempo el cliente realizó una petición. Se mide como el tiempo en milisegundos transcurrido desde el uno de enero de 1970.
int getMaxInactiveInterval():	devuelve el máximo tiempo de inactividad, medido en segundos, que se va a permitir para esta sesión. Si la sesión está inactiva más de ese tiempo, será destruida por el contenedor.
ServletContext getServletContext():	devuelve el ServletContext al cual pertenece esta sesión.
void invalidate():	invalida la sesión y elimina los objetos que estuviesen ligados a ella.
void removeAttribute(String name):	elimina de la sesión el atributo asociado al nombre que se le pasa como parámetro.
void setMaxInactiveInterval(int interval):	especifica el tiempo en segundos que el contenedor esperará antes de invalidar esta sesión.

Servlets: Sesiones y Cookies

- Sesiones en Java (Session Tracking)
  1) Primero se debe llamar al método getSession()
  2) Recuperar algo almacenado en la sesión
  3) Sino esta ,se crea
  4) Finalmente se añade a la sesión el objeto a almacenar

```
protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException {

        response.setContentType("text/html;charset=UTF-8");
        String valor = request.getParameter("num");
        if (valor != null) {
           Integer i = new Integer(Integer.parseInt(valor));
           HttpSession s = request.getSession();
           ArrayList<Integer> l =(ArrayList<Integer>)s.getAttribute("lista");
           if (l==null)
                l = new ArrayList<Integer>();
           l.add(i);
           s.setAttribute("lista", l);
           response.getWriter().append("Hay "+l.size()+" datos");
        } else {
           response.getWriter().append("Faltan datos");
        }

}
```

http://localhost:8080/EjemplosClase/Seguimiento/Sesiones?num=20

Servlets: Sesiones y Cookies

- Sesiones en Java (Session Tracking)
  - La sesión en sí es un objeto Java, que por tanto ocupa memoria.
  - Para evitar problemas de memoria se debe configurar en el descriptor de despliegue de la aplicación cuánto tiempo en minutos deseamos mantener vivas las sesiones de usuario.
    - ✓ Es recomendable limpiar las sesiones cuando un usuario se desconecta.
  - Se emplea la siguiente etiqueta en el descriptor de despliegue:

```
<session-config>
  <session-timeout>
        15
  </session-timeout>
</session-config>
```

Servlets: Sesiones y Cookies

- Campos ocultos de formularios
  - Incluir campos ocultos con los datos a mantener
    - `<INPUT type=“hidden” name=“session” value=“1234”>`
  - Ventajas
    - Funciona incluso si las Cookies no son soportadas o están desactivadas
  - Desventajas
    - Cantidad de procesamiento tedioso
    - Todas las páginas deben ser el resultado de envios de formularios
  - Ejemplo: `ShoppingCart.java`

Servlets: Sesiones y Cookies

- Reescritura de URLs
  - El cliente añade ciertos datos extra que identifican la sesión al final de cada URL
    - `http://host/path/servlet/name?jsessionid=1234`
  - El servidor asocia ese identificador con datos que ha guardado acerca de la sesión
  - Ventajas
    - Funciona incluso si las Cookies no son soportadas o están desactivadas
  - Desventajas
    - Se deben codificar todas las URLs referentes al sitio propio
    - Todas las páginas deben generarse dinámicamente
    - Funciona mal para links desde otros sitios
  - Ejemplo: `SessionSnoop.java`

Servlet: Gestión de Errores

- En las aplicaciones Java EE tenemos dos formas de gestionar los posibles errores que sucedan: declarativamente o programáticamente.
  - El modo declarativo consiste en añadir las etiquetas adecuadas al descriptor de despliegue de la aplicación e indicar en las etiquetas cómo se va a gestionar la excepción: mostrando un recurso estático, o invocando a un Servlet.
    - ✓ Ventaja: Más flexible y potente
    - ✓ Desventaja: A veces tedioso y mezcla código con excepciones
  - El modo programático consiste en dentro del código fuente de nuestro Servlet, incluir el código fuente requerido para la gestión de los errores, o incluir redirecciones al recurso estático o dinámico que va a gestionar la excepción.
    - ✓ Ventajas: Simpleza y separación
    - ✓ Desventaja: Menos flexible y potente

Servlet: Gestión de Errores

- Gestión de códigos de errores HTTP
  - En web.xml se puede incluir una o varia etiquetas <error-page>.
  - Dentro de cada una de estas etiquetas, se indica un recurso estático o dinámico destinado a gestionar el error http correspondiente de la aplicación

```
<error-page>
        <error-code>404</error-code>
        <location>404error.html</location>
</error-page>
<error-page>
        <error-code>500</error-code>
        <location>500error.html</location>
</error-page>
```

  - Aunque estos errores son gestionados por el servidor de aplicaciones, es bueno redefinirlo para que de esta forma, proporcionar una salida más amigable

Servlet: Gestión de Errores

- Gestión declarativa de excepciones
  - En el fichero web.xml se establecen las distintas excepciones y las acciones a realizar (de forma genérica independientemente al servlet que la ha producido)

```
<error-page>
  <exception-type>NullPointerException</exception-type>
  <location>/GestorDeExcepcionesServlet</location>
</error-page>
<error-page>
  <exception-type>NumberFormatException</exception-type>
  <location>/GestorDeExcepcionesServlet</location>
</error-page>
```

  - Acción:

```
protected void processRequest(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
  Exception exception = (Exception)request.getAttribute("javax.servlet.error.exception");
  String urlFuenteDelError = (String)request.getAttribute("javax.servlet.error.request_uri");
  response.setContentType("text/html;charset=UTF-8");
  PrintWriter out = response.getWriter();
  try {
  out.println("<html>");
  …....
  Exception.printStrackTrace(out);
```

Servlet: Gestión de Errores

- Gestión programática de excepciones
  - En el código fuente es donde se realiza la captura de las excepciones y se toman las acciones a realizar.
  - Esta opción es mucho más configurable

```
@WebServlet("/Autenticacion3")
public class Autenticacion3 extends HttpServlet {
  private String user = null;
  private String password = null;
  public void init(ServletConfig config) throws ServletException {
    super.init(config);
     ServletContext context = getServletContext();
     String ficheroUsuarios = context.getInitParameter("ficheroUsuarios");
     InputStream is = context.getResourceAsStream(ficheroUsuarios);
     Properties ficheroPropiedades = new Properties();
    try {
         ficheroPropiedades.load(is);
         user= ficheroPropiedades.getProperty("UserAdmin");
         password = ficheroPropiedades.getProperty("PasswordAdmin");
     } catch (Exception ex) {
         Logger.getLogger(Autenticacion3.class.getName()).log(Level.SEVERE,
                  "No se pudo cargar el fichero con los password", ex);
     }
}
```

Concurrencia en Servlet

- Un contenedor permite el acceso de múltiples thread que están ejecutando de modo concurrente el mismo código.
- Por este motivo, en ocasiones el programador debe escribir un programa teniendo en cuenta consideraciones de concurrencia.
- Que datos no generan problemas de concurrencia (thread-safe)
  - Las variables locales de los métodos están confinadas en el stack, y cada thread tiene su propio stack
  - Cualquier objeto que esté almacenado en la petición (HttpServletRequest)
- Cuales sí:
  - Variables estáticas definidas dentro de la clase del Servlet
  - Objetos que hayas guardado como atributos del contexto
  - Objetos que se guardan en la sesión del usuario (dos lenguetas de un navegador)

Concurrencia en Servlet

- La solución es emplear sincronización manual sobre las variables que puedan provocar estos problemas (synchronized)

```
protected void processRequest(HttpServletRequest request,
    HttpServletResponse response) throws ServletException, IOException {

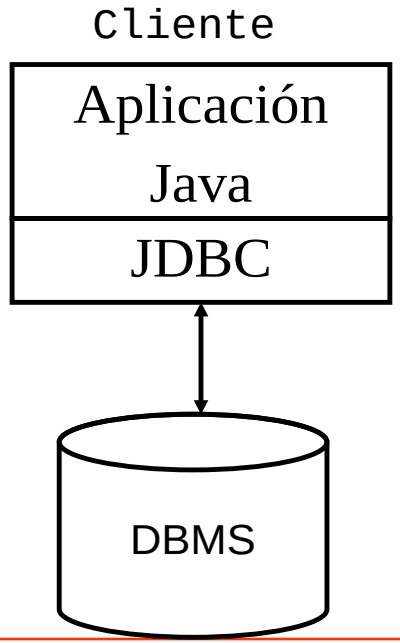
  synchronized (variable) {

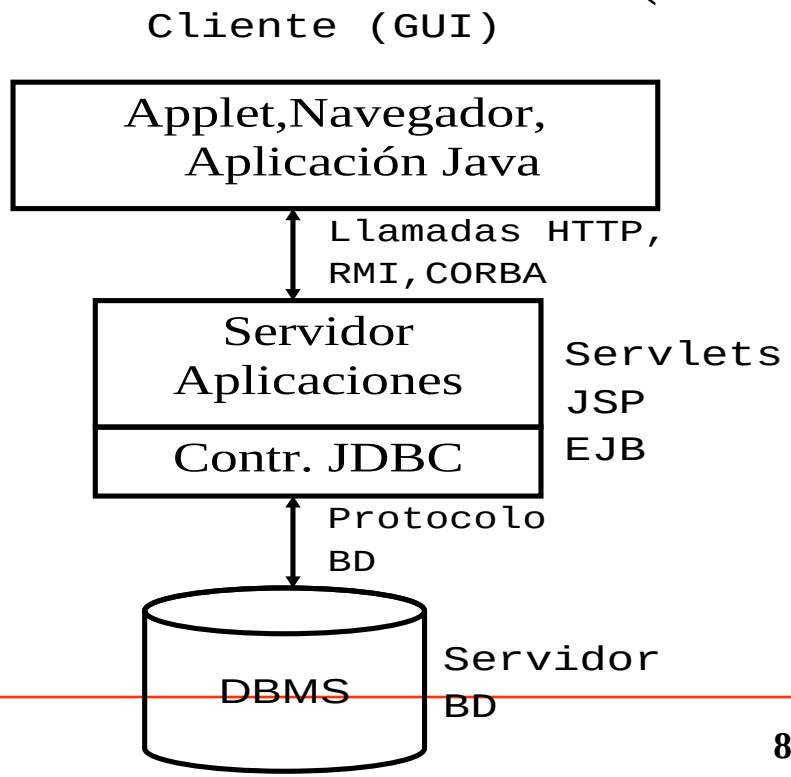
            <<operaciones>>

  {
```

Servlet y JDBC

- Un gran número de aplicaciones web utilizan una base de datos para realizar la persistencia de los datos.
- Existe principalmente tres soluciones
  - Utilizar directamente la API de conexión JDBC
  - Utilizar un pool de conexiones
  - Utilizar un motor de persistencia como JPA o Hibernate (MDAI)





Servlet y JDBC

- Pasos para programar en JDBC
  - Cargar el driver de la base de datos a acceder
  - Definir la URL de la BD
  - Establecer la conexión
  - Crear una orden SQL
  - Ejecutar y Procesar los resultados
  - Cerrar la conexión

Servlet y JDBC

- Pasos para programar en JDBC
  - Cargar el driver de la base de datos a acceder
  - Definir la URL de la BD
  - Establecer la conexión
  - Crear una orden SQL
  - Ejecutar y Procesar los resultados
  - Cerrar la conexión

- Pasos básicos con más detalle:
  - **Cargar Driver.** El método ***forName*** de ***Class*** carga el driver JDBC específico e inicializa ***DriverManager***.
  - **Conectar con la BD.** ***getConnection*** en ***DriverManager*** retorna un objeto ***Connection.*** El objeto ***Connection*** es usado para comunicar con la BD especificada en ***getConnection***.
  - **Ejecutar SQL.** ***createStatement*** en ***Connection*** retorna un objeto ***Statement*** el cual puede ser usado para mandar sentencias **SQL** a la BD conectada
  - Usando ***executeQuery*** o ***executeUpdate*** en ***Statement*** permite o consultar o modificar la BD. (Búsqueda retorna un objeto ***ResultSet*** object.
  - **Procesar** el ***ResultSet*** o los posibles errores que se produzcan
  - Con el método **Close** de ***Connection*** se cierra la conexión.

Servlet y JDBC

- Pasos para programar en JDBC
  - Cargar el driver de la base de datos a acceder
    - Class.forName(“com.informix.jdbc.IfxDriver");
    - Class.forName(“com.novell.sql.LDAPDriver”);
    - Class.forName("com.mysql.jdbc.Driver");
  - Ejemplo de URL
    - jdbc:mysql://localhost/test (base de datos local)
    - jdbc:mysql://158.49.96.41:8088/trafico (base de datos en Mexis)
    - jdbc:oracle://ServerOracle
    - jdbc:odbc:mydatabase (Conexión por ODBC)
    - jdbc:informix-sqli://hostname:port/ dbname:INFORMIXSERVER=serverName (para las bases CGE)
    - jdbc:ldap://server;baseDN=baseDN;useCleartext=true (Para BD en ldap)

Cargar Driver, Definir URL y establecer conexión

```
package Clase;
import java.sql.*;
public class ProbandoConexion01 {
    Connection con;
    Statement st;
    ResultSet rs;
    public void abrirConexion() {
        try {
            String userName="ljarevalo";
            String password="??????????";
            String url="jdbc:oracle:thin:@//158.49.96.163:1521/ORCL";
            Class.forName("oracle.jdbc.OracleDriver").newInstance();
            con = DriverManager.getConnection(url, userName, password);
            System.out.println("Conexion a la BD");
        } catch (Exception e) {System.out.println("Error en conexiobn
");}
    }
     //Para cerrar la conexion una vez terminadas las consultas
    public void cerrarConexion() {
        try {
            con.close();
            System.out.println("Conexion cerrada");
        } catch (SQLException e) {System.out.println("Error al cerrar
conexion");}
    }
    public static void main(String[] args) {
        ProbandoConexion01 c= new ProbandoConexion01();
        try {
            c.abrirConexion();
            c.cerrarConexion();
        } catch(Exception e) {

        }
    }
```

URL de la conexión

Cargar driver

Establecer conexión

Cerrar conexión

Ejecutar consulta: Statement

```
package Clase;
import java.sql.*;
public class ProcesarResultados {
    Connection con;
    Statement st;
    ResultSet rs;
    public void abrirConexion() {
          ............
    }
//Para cerrar la conexión una vez terminadas las consultas
    public void cerrarConexion() {
          ............
    }
    public void obtenerDatosTabla(String Query) {
        try {
            st =con.createStatement();
            rs = st.executeQuery(Query);
            System.out.println("Tabla abierta");
        } catch (SQLException e) {
            System.out.println("Error al Abrir tabla ");
        }
    }

    public static void main(String[] args) {
        ProcesarResultados c= new ProcesarResultados();
        try {
            c.abrirConexion();
            String Query="SELECT * FROM Modelos";
            c.obtenerDatosTabla(Query);
            c.cerrarConexion();
        } catch(Exception e) {

        }
    }
}
```

Ejecutar consulta

Procesar resultados: ResultSet

```
package Clase;
import java.sql.*;
public class ProcesarResultados {
    Connection con;
    Statement st;
    ResultSet rs;
    ......
    //Se procesa por el nombre de los atributos
    public void ProcesarResultados_At() {
        try {
            while (rs.next()) {
                byte CodMarca = rs.getByte("COD_MARCA");
                byte Cod_Modelo= rs.getByte("COD_MODELO");
                String nombre = rs.getString("NOM_MODELO");
                String potencia = rs.getString("POTENCIA");
                System.out.println(String.valueOf(CodMarca)+ ", " +
String.valueOf(Cod_Modelo)+ ", " + nombre + " , "+potencia);
            }
        } catch (Exception e) { System.out.println("Error al visualizar datos");}
    }
    //Se  procesa por el campo recuperado
    public void ProcesarResultados_Pos() {
        try {
            rs.beforeFirst(); //Para volver al principio del cursor
            while (rs.next()) {
                byte CodMarca = rs.getByte(1);
                byte Cod_Modelo= rs.getByte(2);
                String nombre = rs.getString(3);
                String potencia = rs.getString(4);
                System.out.println(String.valueOf(CodMarca)+ ", " +
String.valueOf(Cod_Modelo)+ ", " + nombre + " , "+potencia);
            }
        } catch (Exception e) { System.out.println("Error al visualizar datos"); }
    }
    public static void main(String[] args) {
      ........
    }
}
```

Procesar resultados

Servlet y JDBC

- Solución 1: Utilizar directamente la API de conexión JDBC
  - Se debe realizar durante la carga del servlet (init): driver JDBC, la URL, el establecimiento de la conexión y el statement → evitar coste computacional de realizarlo constantemente

```
public class JDBCServlet extends HttpServlet {

  private Statement sentencia = null;
  private Connection conexion = null;

  @Override
  public void init(ServletConfig config) {
     try {
         Class.forName("com.mysql.jdbc.Driver");
         conexion = DriverManager.getConnection(
                  "jdbc:mysql://localhost:3306/msn", "ljarevalo",
                  "????");
         sentencia = conexion.createStatement();
      } catch (ClassNotFoundException ex) {
         Logger.getLogger(JDBCServlet.class.getName()).log(Level.SEVERE,
                  "No se pudo cargar el driver de la base de datos", ex);
      } catch (SQLException ex) {
         Logger.getLogger(JDBCServlet.class.getName()).log(Level.SEVERE,
                  "No se pudo obtener la conexión a la base de datos", ex);
  }
}
```

Proceso de carga

Solución 1: Utilizar directamente la API de conexión JDBC

```
public void doGet(HttpServletRequest req, HttpServletResponse res)
          throws ServletException, IOException {
  ResultSet resultados = null;// Objeto para guardar los resultados
  res.setContentType("text/html");
  PrintWriter out = res.getWriter();
  try {
      synchronized (sentencia) {
          resultados = sentencia.executeQuery("SELECT * FROM usuarios");
       }
       out.println("<html>\n <body>\n <h1>Usuarios</h1>");
       // Mostramos las distintas asignaturas
      while (resultados.next()) {
          out.println("<br>Nombre=" + resultados.getString("nombre")
                    + " Nick=" + resultados.getString("nick")
                    + " Password=" + resultados.getString("password"));
       }// Fin while
       // Escribimos el final de la pagina
       out.println("</body>");out.println("</html>");
   } catch (SQLException e2) {
          out.println("ERROR:Fallo en SQL: " + e2.getMessage());
   } finally {
          if (resultados != null) { // Se cierra resultSet
              try {
                    resultados.close();
               } catch (SQLException ex) {
                    Logger.getLogger(JDBCServlet.class.getName()).log(
                              Level.SEVERE, "No se pudo cerrar el Resulset", ex);
               }
          }
   } }
```

Sincronizado

Servlet y JDBC

- Solución 1: Utilizar directamente la API de conexión JDBC
  - La conexión, generada en init(), es necesario liberarla cuando el Servlet es destruido por el contenedor → método destroy()

Redefinición método destroy()

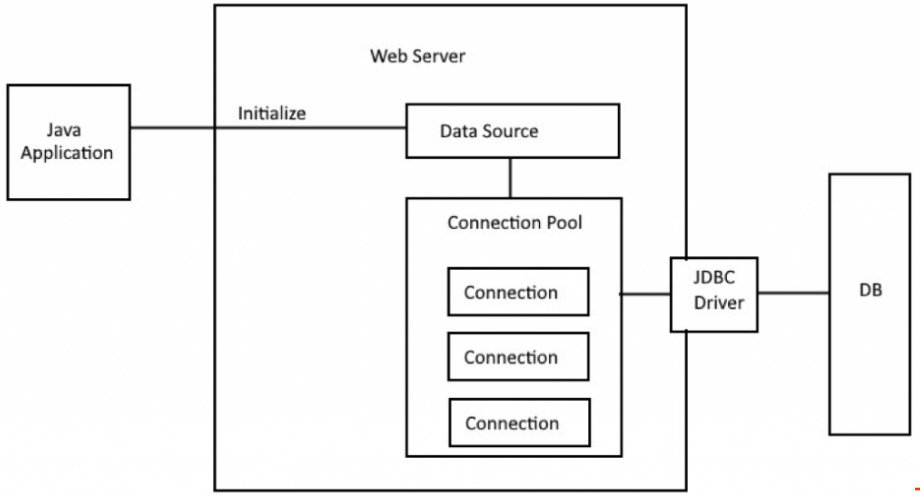
```
@Override
public void destroy() {
  try {
          sentencia.close();
   } catch (SQLException ex) {
          Logger.getLogger(JDBCServlet.class.getName()).log(Level.SEVERE,
                  "No se pudo cerrar el objeto Statement", ex);
   } finally {
          try {
                  conexion.close();
          } catch (SQLException ex) {
              Logger.getLogger(JDBCServlet.class.getName()).log(Level.SEVERE,
                  "No se pudo cerrar el objeto Conexion", ex);
          }
   }
}
```

Servlet y JDBC

- Solución 1: Utilizar directamente la API de conexión JDBC
  - Desventaja: aunque se ha usado sincronización solamente se tiene una única conexión a la base de datos, y por tanto sólo es posible ejecutar de modo simultáneo una única consulta contra ella.
  - Si varios usuarios están empleando la aplicación, todas esas consultas acabarán en una cola y se irán ejecutando de una en una.
  - Solución óptima → tener varias conexiones en paralelo → **Pool de Conexiones** de modo que cuando llega una petición http, empleamos una conexión que no se esté usando en ese momento
    - ✓ El pool de conexiones esta gestionado por el propio servidor.

Servlet y JDBC

- Solución 2: Pool de conexiones → **Pasos en Tomcat**:
  - Configurar el archivo web.xml de nuestra aplicación para que acceda al origen de datos adecuado.
  - Configurar el archivo context.xml con los parámetros de la conexión.
  - En el código java de nuestra aplicación:
    - ✓ Crear el objeto DataSource
    - ✓ Obtener una conexión disponible de dicho objeto.



- Solución 2: Pool de conexiones → Pasos
  - Configurar el archivo web.xml de nuestra aplicación para que acceda al origen de datos adecuado.

```
<resource-ref>
        <description>Tomcat- DataSource</description>
        <res-ref-name>jdbc/TestDB</res-ref-name>
        <res-type>javax.sql.DataSource</res-type>
        <res-auth>Container</res-auth>
        <res-sharing-scope>Shareable</res-sharing-scope>
</resource-ref>
```

  - Configurar el archivo context.xml con los parámetros de la conexión.
  - En el código java de nuestra aplicación:
    - ✓ Crear el objeto DataSource
    - ✓ Obtener una conexión disponible de dicho objeto.

- Solución 2: Pool de conexiones → Pasos
  - Configurar el archivo web.xml de nuestra aplicación para que acceda al origen de datos adecuado.
  - Configurar el archivo context.xml con los parámetros de la conexión.
    - En la carpeta /META-INF de nuestra aplicación se debe crear el archivo context.xml con los parámetros de la conexión:

```
<Resource name="jdbc/TestDB" auth="Container"
            type="javax.sql.DataSource" username="usuario" password="pwd_de_la_bdd"
            driverClassName="com.mysql.jdbc.Driver"
            url="jdbc:mysql://servidor_bdd/nombre_bdd"
            maxActive="100"  maxIdle="30" maxWait="10000"
            />
```

  - En el código java de nuestra aplicación:
    - Crear el objeto DataSource
    - Obtener una conexión disponible de dicho objeto.

Servlet y JDBC

- Solución 2: Pool de conexiones → Pasos
  - Configurar el archivo web.xml de nuestra aplicación para que acceda al origen de datos adecuado.
  - Configurar el archivo context.xml con los parámetros de la conexión.
  - En el código java de nuestra aplicación:
    - ✓ Crear el objeto DataSource y obtener una conexión disponible

```
public void init(ServletConfig config) throws ServletException {
  DataSource servicioConexiones=null;
  try     {
   // Referencia al servicio de conexiones
     Context envCtx = (Context) new InitialContext().lookup("java:comp/env");
     servicioConexiones = (DataSource)envCtx.lookup("/jdbc/TestDB");
     conexion = servicioConexiones.getConnection();
     sentencia = conexion.createStatement();
  } catch (Exception e){
   throw new ServletException( "Imposible recuperar java:comp/env/jdbc/testDB", e);
}
}
```

- Solución 2: Pool de conexiones → Pasos
  - ✓ auth=”Container” (tipo, en este caso es contenedor)
  - ✓ driverClassName=”com.mysql.jdbc.Driver” (driver a cargar por Tomcat)
  - ✓ maxActive=”100″ (máximo de conexiones activas, por defecto)
  - ✓ maxIdle=”100″ (máximo de conexiones, por defecto)
  - ✓ maxWait=”10000″ (máximo de tiempo en mili-segundos para la conexión, por defecto)
  - ✓ name=”jdbc/test” (el nombre del contenedor)
  - ✓ password=”123456″ (contraseña de la base de datos)
  - ✓ type=”javax.sql.DataSource” (señala datasource para la conexión)
  - ✓ url=”jdbc:mysql://localhost:3306/test” (url de la base de datos)
  - ✓ username=”root” (usuario de la base de datos)
  - ✓ Password=”” (contraseña)

Configuración de BD

- Bases de Datos relacionales
  - Se trata de bases de datos de servidor con soporte ACID y un funcionamiento autónomo.
- Bases de datos embebidas Java (https://en.wikipedia.org/wiki/Embedded_database)
  - Se trata de bases de datos que carecen de servidor, están incrustadas en la propia aplicación y suelen estar almacenadas en ficheros locales.
  - Suelen tener un modo de funcionamiento en el que mantienen los datos en memoria → rendimiento muy alto.
  - Otra ventaja es que no tenemos que encargarnos de mantener y gestionar un servidor de bases de datos.
  - Suele usarse para ejecutar los tests de integración contra una base de datos real, aunque no física.

Configuración de BD

- Las BD relacionales más habituales son:
  - MySQL
  - PostGreSQL
  - MariaDB
  - Oracle
  - Microsoft SQLServer
  - ….











Configuración de BD

- Las BD embebidas más habituales son:
  - SQLite: https://www.sqlite.org/
  - HSQLDB: http://hsqldb.org/
  - H2, http://www.h2database.com/html/main.html
  - Derby, http://db.apache.org/derby/









Configuración de BD

- SQLite
  - Se trata de una librería escrita en ANSI C, de menos de 500KB, multi-plataforma, sin dependencias externas, almacena todo el contenido de la base de datos en un solo fichero.
  - Se puede utilizar desde C y C++, pero también desde otros lenguajes de programación (PHP, Python, Java, .NET ...).
  - En el caso de Java podemos gestionar esta base de datos a través de JDBC.
  - Es compatible con ACID
  - En su versión 3, SQLite permite bases de datos de hasta 2 Terabytes de tamaño, y también permite la inclusión de campos tipo BLOB.
  - Utilizado por Firefox, Adobe Photoshop, XBMC, Skype, etc.

Configuración de BD

- Derby o JavaDB
  - JavaDB es una distribución de Oracle de la base de datos de código libre Derby.
  - Soporta el estándar ANSI/ISO SQL a través de JDBC y Java EE. Estas librerías están incluidas en el JDK.
  - Almacena la base de datos en multiples archivos, lo que puede resultar útil para escalar el almacenamiento.
  - Solo podemos utilizarlo en Java, no desde otros lenguajes.
  - Posee tres productos asociados a la marca:
    - ✓ Derby Embedded Database Engine: El motor propiamente dicho.
    - ✓ Derby Network Server: Permite convertir Derby en una base de datos que sigue el modelo cliente-servidor tradicional.
    - ✓ Database Utilities: Un paquete de utilidades.

Configuración de BD

- H2
  - H2 es una base de datos SQL Java.
  - H2 opera en tres modos:
    - Embebido es el tradicional, en el que el servidor gestiona los datos en archivos, teniendo una pequeña parte en memoria.
    - En memoria, aloja todos los datos en memoria, con lo que se consigue una velocidad increíble en el acceso a los datos.
    - Servidor: Permite configurarlo como un servidor físico
    - Modo de servidor web: HSQLDB puede actuar como un servidor web, aceptando consultas de SQL mediante HTTP, o puede ejecutarse como un servlet dentro de cualquier contenedor web estándar
  - Tamaño pequeño: 1 JAR de 1 Mb
  - Ofrece soporte para las características ACID

Configuración de BD

- HSQL: Hyperthreaded Structured Query Language Database
  - HSQLDB es un sistema gestor de bases de datos libre escrito en Java.
  - La suite ofimática OpenOffice.org lo incluye desde su versión 2.0 para dar soporte a la aplicación Base.
  - 4 tipos de ejecución:
    - Modo en memoria
    - Modo autónomo: La aplicación crea una conexión de base de datos utilizando JDBC y el motor de HSQLDB se ejecuta dentro de la aplicación
    - Modo de servidor: Esta es la configuración estándar de la base de datos de cliente/servidor similar a otras bases de datos relacionales
    - Modo de servidor web: HSQLDB puede actuar como un servidor web, aceptando consultas de SQL mediante HTTP, o puede ejecutarse como un servlet dentro de cualquier contenedor web estándar

Configuración de BD

- Comparativa obtenida de http://www.h2database.com/html/main.html

Feature	H2	Derby	HSQLDB	MySQL	PostgreSQL
Pure Java	Yes	Yes	Yes	No	No
Embedded Mode (Java)	Yes	Yes	Yes	No	No
In-Memory Mode	Yes	Yes	Yes	No	No
Explain Plan	Yes	Yes *12	Yes	Yes	Yes
Built-in Clustering / Replication	Yes	Yes	No	Yes	Yes
Encrypted Database	Yes	Yes *10	Yes *10	No	No
Linked Tables	Yes	No	Partially *1	Partially *2	No
ODBC Driver	Yes	No	No	Yes	Yes
Fulltext Search	Yes	Yes	No	Yes	Yes
Domains (User-Defined Types)	Yes	No	Yes	Yes	Yes
Files per Database	Few	Many	Few	Many	Many
Row Level Locking	Yes *9	Yes	Yes *9	Yes	Yes
Multi Version Concurrency	Yes	No	Yes	Yes	Yes
Multi-Threaded Statement Processing	No *11	Yes	Yes	Yes	Yes
Role Based Security	Yes	Yes *3	Yes	Yes	Yes
Updatable Result Sets	Yes	Yes *7	Yes	Yes	Yes
Sequences	Yes	Yes	Yes	No	Yes
Limit and Offset	Yes	Yes *13	Yes	Yes	Yes
Window Functions	No *15	No *15	No	No	Yes
Temporary Tables	Yes	Yes *4	Yes	Yes	Yes
Information Schema	Yes	No *8	Yes	Yes	Yes
Computed Columns	Yes	Yes	Yes	No	Yes *6
Case Insensitive Columns	Yes	Yes *14	Yes	Yes	Yes *6
Custom Aggregate Functions	Yes	No	Yes	Yes	Yes
CLOB/BLOB Compression	Yes	No	No	No	Yes
Footprint (jar/dll size)	~1.5 MB *5	~3 MB	~1.5 MB	~4 MB	~6 MB

Bibliografía

- Tutorial básico de java EE. Abraham Otero. JavaHispano
  - http://www.javahispano.org/portada/2011/7/28/tutorial-basico-de-java-ee-por-abraham-otero.html
- Tutorial de Servlets de Sun:
  - http://java.sun.com/j2ee/tutorial/1_3-fcs/doc/Servlets.html
- **Libros Editorial eni**:
  - Jakarta EE- Desarrolle aplicaciones web en Java
  - Java Enterprise Edition - Desarrollo de aplicaciones web con JEE 6
- Libro Gratuito sobre Servlets y JSP (muy completo). Buscar en google: books addisonwesley ServletsJSP
- Sitio web dedicado a la programación con servlets y JSP:
  - http://www.moreservlets.com/